FELADATKIÍRÁS

Ennek az oldalnak a helyére a feladatkiírást kell beszúrnod az elektronikus változatban (praktikusan egy képernyőkép formájában). A nyomtatott változatban ennek az oldalnak a helyére a diplomaterv portálról letöltött, jóváhagyott feladatkiírást kell befűzni. A feladatkiírás tanszéki aláírás és pecsét nélkül is érvényes.

Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Automatizálási és Alkalmazott Informatikai Tanszék

Horváth Gellért

AI-TÁMOGATOTT TÁRGYFELISMERÉS ÉS HASONLÓSÁGVIZSGÁLAT MOBILALKALMAZÁSBAN

KONZULENS

DR. Ekler Péter

BUDAPEST, 2025

Tartalomjegyzék

Összefoglaló **6**
Abstract **7**
1 Bevezetés **8**
1.1 Motiváció 8
1.2 Az alkalmazás felhasználói nézőpontból 11
1.2.1 Név és ikon 11
1.2.2 Felhasználói funkciók 11
2 Alkalmazott technológiák **16**
2.1 PostgreSQL 16
2.2 ASP.NET Core (backend) 16
2.2.1 Web API 17
2.2.2 Dependency Injection 19
2.3 Entity Framework 20
2.3.1 Objektum-relációs leképezés 20
2.3.2 Navigációs tulajdonságok 22
2.4 React Native 23
2.4.1 JSX Tagek 23
2.4.2 Expo 24
2.4.3 React Native Elements 24
2.5 Python Service FastAPI 25
2.6 Tárgyfelismerés 25
2.6.1 Label Studio 26
2.6.2 YOLOv8 26
2.7 Hasonlóságvizsgálat 28
2.7.1 Embedding 29
2.7.2 Metric Learning 31
3 Architektúra **33**
3.1 Mappastruktúra 33
3.2 Adatbázisséma 35
3.2.1 Users, UserContacts 36
3.2.2 CollectionTypes, Attributes, ItemOptions, ItemAttributes 37

3.2.3 Collections, Follows, Reactions 39
3.2.4 Items, ItemTags 40
3.2.5 Trades, TradeItems, Offers, OfferItems, TradeReviews 41
3.3 Vektor adatbázisséma 42
3.4 Betanított modellek konvenció 43
3.5 „Univerzális” megoldás összefoglalás 43
4 Projekt részletezése 44
4.1 ASP.NET Core backend 44
4.1.1 Third party bejelentkezés, és JWT szolgáltatás 44
4.1.2 Generikus CRUD 49
4.2 React Native 53
4.2.1 Expo app.json 53
4.2.2 Expo router 55
4.2.3 Zustand global store 56
4.3 Python service, modell betanítás 57
4.3.1 Tárgyfelismerő modell betanítás 57
4.3.2 Embedding modell betanítás 59
4.3.3 Tárgyfelvétel csővezeték 63
5 Projektértékelés 68
6 Összefoglaló 69
Irodalomjegyzék 70
Függelék A – Képernyőképek az applikációból 72
Függelék B – Teljes ER diagram 73

HALLGATÓI NYILATKOZAT

Alulírott **Horváth Gellért** szigorló hallgató kijelentem, hogy ezt a szakdolgozatot meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző, cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövegét pedig az egyetem belső hálózatán keresztül (vagy hitelesített felhasználók számára) közzétegye. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Kelt: Budapest, 2025. 10. 19

...…….

Horváth Gellért

Összefoglaló

A szakdolgozat, vagy diplomaterv elkészítése minden egyetemi hallgató életében egy fontos mérföldkő. Lehetőséget ad arra, hogy az egyetemi évei során megtanultakat kamatoztassa és eredményeit szélesebb közönség előtt bemutassa, s mérnöki rátermettségét bizonyítsa. Fontos azonban, hogy a dolgozat elkészítésének folyamata számos csapdát is rejt magában. Rossz időgazdálkodás, hiányos szövegszerkesztési ismeretek, illetve a dolgozat készítéséhez nélkülözhetetlen „műfaji" szabályok ismeretének hiánya könnyen oda vezethetnek, hogy egy egyébként jelentős időbefektetéssel készült kiemelkedő szoftver is csak gyengébb minősítést kapjon a gyenge minőségű dolgozat miatt.

E dokumentum – amellett, hogy egy általános szerkesztési keretet ad a dolgozatodnak – összefoglalja a szakdolgozat/diplomaterv írás írott és íratlan szabályait. Összeszedjük a Word kezelésének legfontosabb részeit (címsorok, ábrák, irodalomjegyzék stb.), a dolgozat felépítésének általános tartalmi és szerkezeti irányelveit. Bár mindenkire igazítható sablon természetesen nem létezik, megadjuk azokat az általános arányokat, oldalszámokat, amelyek betartásával jó eséllyel készíthetsz egy színvonalas dolgozatot. A részletes és pontokba szedett elvárás-lista nem csupán a dolgozat írásakor, de akár más dolgozatok értékelésekor is kiváló támpontként szolgálhat.

Az itt átadott ismeretek és szemléletmód nem csupán az aktuális feladatod leküzdésében segíthet, de hosszútávon is számos praktikus fogással bővítheti a szövegszerkesztési és dokumentumkészítési eszköztáradat.

Abstract

Ide jön a ½-1 oldalas angol nyelvű összefoglaló, amelynek szövege a Diplomaterv Portálra külön is feltöltésre kerül. Ez a magyar nyelvű összefoglaló angolra fordított változata.

1 Bevezetés

1.1 Motiváció

Egy szakdolgozat projekt mindig a feladat megálmodásával kezdődik. Ez a „beugró” rész sok hallgatónál a teljes projekt egyik legnehezebb része lehet, ugyanis a hallgatónak meg kell találnia a számos téma közül azt, amin szívesen dolgozik, végig fenntartja az érdeklődését, továbbá objektív szemléletet is alkalmaz, vagyis igyekszik olyan témát választani aminek helye lehet a modern szoftverpiacon, új dolgokat tanul belőle, viszont nem vállalja túl magát. Ez a döntés rendkívül fontos, ennek terhe a teljes féléven végig kísér.

Számomra a téma kiválasztása egyszerű volt, segített benne az, hogy egyből találtam hozzá személyes, illetve a szakmámhoz köthető motivációt is. A személyes motivációm kiindulópontja a hobbim volt. Hobbi szinten gyűjtöm az üveges üdítők és sörök fémkupakjait. Nem tudom, hogy más gyűjtemények tulajdonosai hogyan tartják számon a több száz darabból álló kollekcióikat, de nekem ez igen nehezen ment. Sokszor már a megvétel előtt sem tudtam eldönteni, melyik üdítőt vagy sört válasszam - az elfogyasztása mellett azzal a céllal, hogy az új fémkupak bekerüljön a gyűjteményembe. Lényegében ez a probléma volt az, ami a személyes inspirációt adta a téma kiválasztásához. A személyes motivációm ismertetése már utalhatott arra, hogy a projektem egy kollekció, gyűjteménynyilvántartó rendszer megvalósítása. Eredetileg az alkalmazást kizárólag a kupakgyűjtemények kezelésére terveztem, azonban a konzulensemmel egyeztetve arra jutottunk, hogy az alkalmazás szélesebb felhasználói bázist érhetne el, ha univerzális megoldást kínálna. Ennek megfelelően a feladatot kibővítettem, így az új cél egy általános kollekciókezelő rendszer megvalósítása lett. Fontos tisztázni, mit jelent az „univerzális megoldás”. Ez azt jelenti, hogy a felhasználók a kollekció létrehozásakor többféle kollekciótípus közül választhatnak, amelyeket a rendszer teljes körűen támogat. Emellett új kollekciótípusok támogatása is gyorsan és egyszerűen hozzáadható a programhoz anélkül, hogy a meglévő kódot módosítani kellene. Bár a program alapvető célja már világos, eddig semmi rendkívüli nincs benne: a leírás akár egy hagyományos könyvtárkezelő alkalmazásra is vonatkozhatna.

Az én kupak gyűjteményem írásban nehezen dokumentálható. Ezt egy példán keresztül szemléltetem: egy kupakot sokkal nehezebb virtuális kollekcióba felcímkézni

és adatokkal ellátni, mint egy fociskártyát. A fociskártyák általában egyediek: egy játékoshoz egy lap tartozik, vagy esetenként több, amelyek ritkasági kategóriákba vannak sorolva. A lényeg az, hogy 1-2 tulajdonság alapján egyértelműen beazonosíthatók, és ezeket a tulajdonságokat maga a kártya is egyértelműen megmutatja. Egy kupak ezzel szemben csak több tulajdonság alapján azonosítható pontosan - ilyen például a márka, a márkán belüli típus, az esetleges évjárat vagy egy különkiadás. A beazonosítást tovább nehezíti, hogy ezek az adatok sok kupakon nincsenek egyértelműen feltüntetve, vagyis nem olvashatók le közvetlenül - ezek kiderítése további utánajárást igényel. Pontosan ezért, számomra, és más gyűjtők számára, akiknek a gyűjteményébe a tárgyak nehezen dokumentálhatóak, egy egyszerű könyvtár alkalmazás nem lenne nagy segítség. Valahogy máshogy szerettem volna dokumentálni a tárgyakat. Kitaláltam, hogy a tárgyak fő tulajdonsága egy fotó legyen róluk. Ez minden korábban említett problémát megold, hiszen egyrészt ez az egy tulajdonság egyértelműen beazonosítja a tárgyat, másrészt manapság egy telefonnal nagyon egyszerűen készíthetünk jó minőségű fényképeket. Így az AI sokkal gyorsabban tudja ellenőrizni, majd mi felvenni az új tárgyakat a kollekcióba, mintha a nehezen kivehető szöveges adatok között kellene keresgélnünk. Természetesen a program úgy lett tervezve, hogy támogassa a felcímkézést is, tehát a képek alapján felvett tárgyakat később adatokkal is kiegészíthetjük. Az előbb már meg is említettem a szakmához köthető motivációkat, az alábbiakban ezeket fejtem ki: az AI és az okostelefonok. Napjaink modern és exponenciálisan fejlődő világában egyre nagyobb teret hódít az AI és annak forradalmi megoldásai, míg az okostelefonok továbbra is a legnépszerűbb elektronikus eszközök közé tartoznak. Egy szakmabelinek ezért elengedhetetlen, hogy legalább alapszinten ismerje ezeket a technológiákat. Ennek tudatában választottam meg a szakmámhoz köthető motivációt. A telefonra való integráció kézenfekvő volt, hiszen a tárgyakról telefonnal lehet fotót készíteni, és ezeket a képeket azonnal továbbítani lehet a telefonon futó alkalmazásnak. Az AI technológia alkalmazása pedig azért vált szükségessé, mert a képet valamilyen módon le kell szűkíteni magára a tárgyra, illetve két fotót meg kell tudni különböztetni egymástól, hogy megállapíthassuk, egy tárgy egyedi-e a kollekcióban. Ezt a funkciót mesterséges intelligencia segítségével lehet megvalósítani. A mesterséges intelligencián alapuló tárgyfelismerés és az egyediségvizsgálat képezi az alkalmazás központi funkcióját; a szakdolgozatom címében szereplő „tárgyfelismerés és hasonlóságvizsgálat” is ezt tükrözi.

Egy valós telefonos alkalmazás rengeteg funkcióval rendelkezik. Én is egy ilyen, valós alkalmazás modellezését tűztem ki célul. Ezért a feladathoz néhány közösségi funkciót is hozzáadtam, hogy a rendszer egy teljes értékű mobilalkalmazást szimuláljon. A plusz funkciók a közösségre épülnek: a felhasználók egy „feed” oldalon láthatják mások új szerzeményeit, feliratkozhatnak más gyűjteményekre, amelyek külön oldalon jelennek meg, valamint a „feed” elsősorban az azonos országból származó bejegyzéseket és a követett kollekciók új tárgyait jeleníti meg. Ezen felül helyet kapott egy csere-bere funkció is, ahol a felhasználók csereajánlatokat tölthetnek fel - akár több tárgyat is egy bejegyzésbe -, és más felhasználók ezekre ajánlatokat küldhetnek, amelyek közül a bejegyzés feltöltője válogathat.

Összefoglalva a motivációk mentén tagolt feladatleírásomat: A projekt célja egy univerzális kollekció- és gyűjteménykezelő rendszer megalkotása, amelynek fő funkciója a tárgyak egyediségének vizsgálata tárgyfelismerésen és hasonlóságvizsgálaton keresztül. A rendszer támogatja a kollekciók létrehozását és szerkesztését, valamint kiegészül közösségi funkciókkal is, mint a „feed” oldal, a kollekciók követése és a csere-bere felület. A teljes projekt így egy komplex, közösségi élményt nyújtó mobilplatformot modellez.

Mostanra tisztán látható, mi a projekt célja, és milyen motivációk vezettek a feladat meghatározásához. Az alábbiakban bemutatom a teljes projektet. Először sorra veszem az alkalmazott technológiákat, a legegyszerűbben megérthetőktől a legösszetettebbek felé haladva. Ezt követően bemutatom az alkalmazás alapjául szolgáló architektúrát, amelyre a backend szerverek épülnek. Ismertetem, miként sikerült elérni az univerzalitást a különböző kollekciótípusok támogatásában. Ezután végigvezetem az olvasót a teljes projekten, és bemutatom annak legfontosabb részeit - mind a backend, mind a frontend oldaláról. A dolgozat végén értékelem a munkámat: megvizsgálom, milyen minőségben sikerült megvalósítani a program komponenseit, és milyen eredményt hozott a fő funkció implementálása. A szakdolgozatot egy rövid összegzéssel zárom, amelyben összefoglalom, hogy a feladatkiírásból mit valósítottam meg, milyen irányokban lehetne továbbfejleszteni a rendszert, mit tanultam a folyamat során, valamint milyen pozitív és negatív tapasztalatokat szereztem a szoftverfejlesztés ezen területén.

1.2 Az alkalmazás felhasználói nézőpontból

1.2.1 Név és ikon

1. ábra Az alkalmazás parancsikonja

Az alkalmazás a *Coffer* nevet kapta, amely a „kincsesláda” angol, kissé egzotikusabb kifejezése. Azért ezt a nevet választottam, mert a kincsesláda szimbolizálja azt a helyet, ahová a felhasználó mindent elhelyez, amit nagy becsben tart – és mi lehetne értékesebb, mint a saját gyűjteménye. Emellett a kincsesláda tágas, így utal arra is, hogy az alkalmazásban számos különféle gyűjtemény elfér. Az alkalmazás ikonját az 1. ábra szemlélteti; ez vizuálisan reprezentálja a kincsesládában őrzött, nagy becsben tartott értékeket.

1.2.2 Felhasználói funkciók

Ebben a fejezetben bemutatom az alkalmazás felhasználói funkcióit, oldalanként haladva. Bár az egyes részek között akadnak összetettebb koncepciók is, igyekszem ezeket röviden, mégis egyértelműen ismertetni. Az oldalakhoz a futó alkalmazásból készült képernyőképek is rendelkezésre állnak, amelyek megtekinthetők az alábbi GitHub Gist oldalon, valamint az A függelékben.

Funkciók bemutatása oldalakra lebontva:

- *Login*: Ha a felhasználó nincs bejelentkezve, ez az oldal szolgál kezdőlapként. Itt kétféle bejelentkezési mód közül választhat: Google- vagy GitHub-fiókkal történő bejelentkezést. Miután a felhasználó igazolta magát valamelyik bejelentkezési mód segítségével, a rendszer ellenőrzi, hogy van-e már Coffer-fiókja. Ha nincs, a regisztrációs oldalra irányítja, ahol meg kell adnia a felhasználónevét és a nemzetiségét. Végül mindkét esetben a kollekciók oldalra jut.

- *Collections*: Ez az oldal listázza a felhasználó összes létrehozott kollekcióját, kollekciótípusonként csoportosítva. Egy saját kollekció kiválasztásával a felhasználó a kollekció kontextusán belül a kezdőlapra, vagyis a *Feed* oldalra kerül. A kollekciók oldalon ezen kívül új kollekció is létrehozható.
- *Feed*: A kollekciókontextuson belüli főoldal. Az oldal tetején egy keresőmező található, amellyel más felhasználókra, kollekcióikra vagy tárgyaikra lehet rákeresni. A keresés akkor ad találatot, ha a beírt szöveg szerepel a felhasználó, a kollekció vagy a tárgy nevében. Tárgyakra *#hashtag* segítségével is kereshetünk. Az oldal ezen kívül feed funkciót is megvalósít: kilistázza a felhasználó számára más felhasználók elmúlt egy hétben szerzett új tárgyait, rangsorolva az alapján, hogy mely kollekciókat követ, majd a közös nemzetiség, végül az egyéb új tárgyak szerint. A sorrendben némi véletlenszerűség is van, hogy a megjelenítés ne legyen túlságosan szabályos. A felhasználó mások új tárgyait értékelheti like formájában, illetve ritkasági szint megadásával. A ritkasági fokozatok: *No Rate, Common, Uncommon, Rare, Relic*. Egy feed bejegyzésben a felhasználóra kattintva a felhasználó, a kollekció nevére kattintva a kollekció, míg a tárgy nevére kattintva a tárgy részletes nézetéhez navigálhatunk el.
- *My Follows*: Ezen az oldalon jelennek meg azok a kollekciók, amelyeket a felhasználó követ. Egy adott kollekcióra kattintva a felhasználó elnavigálhat a kiválasztott kollekció részletes nézetéhez.
- *Barter*: A *Barter* oldal valósítja meg a csere-bere funkciót. A nézet három aloldalból áll. A *Trades* fülön a felhasználó számára releváns „trade”-ek listázódnak. Egy „trade” kiválasztásakor a rendszer elnavigál a „trade” részletes nézetéhez, míg a felhasználóra kattintva a felhasználó részletes profiljára jutunk. A *My Trades* oldal az általunk létrehozott „trade”-eket listázza, aktív és befejezett cserékre bontva. Ezen az oldalon új „trade” is létrehozható: meg kell adnunk a címét, leírását, valamint hogy mit szeretnénk cserébe – más tárgyat vagy pénzt. Ezt követően ki kell választanunk azokat a tárgyakat, amelyeket cserére kínálunk. Fontos megjegyezni, hogy csak olyan tárgyakat választhatunk ki, amelyekből

több darab áll rendelkezésre (vagyis a tárgy *Quantity* értéke nagyobb, mint egy), és amelyek többlete nem szerepel más aktív cserénkben vagy ajánlatunkban (offer). Végül a *My Offers* aloldalon a saját ajánlatainkat láthatjuk az egyes „trade”-ekhez, szintén aktív és befejezett státusz szerint bontva.

- *Trade Details*: Minden „trade”-hez tartozik egy részletes oldal is, amely kontextusfüggő. Ha a felhasználó a saját cseréjének részletes oldalát nézi, akkor azt szerkeszteni és törölni tudja, de csak addig, amíg el nem fogad egy ajánlatot. Ezt követően a csere, illetve az ajánlat csak akkor módosítható bármelyik fél részéről, ha a visszavonást kezdeményező fél küld egy visszavonási kérelmet, és a másik fél azt elfogadja. A saját „trade” részletes nézetében a felhasználó látja az összes beérkezett ajánlatot, és kezelni tudja őket. Az ajánlattevő kontextusában viszont az oldal a csere létrehozójának elérhetőségeit jeleníti meg, valamint a saját ajánlatát - ha már létezik -, vagy lehetőséget biztosít új ajánlat létrehozására.

A „trade” az alábbi állapotokat veheti fel:

Open: bárki küldhet ajánlatot.

Offer Accepted: a felhasználó elfogadott egy ajánlatot; ilyenkor új ajánlat már nem tehető, és a „trade” nem szerkeszthető.

Offer Revert By Creator: a „trade” létrehozója vissza szeretné vonni az elfogadott ajánlatot; ehhez kérelmet kell benyújtania az ajánlattevő felé.

Offer Revert By Offerer: ugyanaz, mint az előző, de az ajánlattevő kezdeményezi.

Traded: a csere lezárult, a felek értékelést írhatnak egymásról.

Az ajánlatok (offers) állapotai hasonlóak, de a kiinduló állapotuk *Pending,* vagyis az ajánlat a csere létrehozójának döntésére vár. Ilyenkor az ajánlat szerkeszthető és törölhető. A *Rejected* állapot azt jelenti, hogy a csere létrehozója elutasította az ajánlatot; ebben az esetben ő nem látja az

ajánlatot, amíg az ajánlattevő újra nem posztolja. Ha az ajánlat elfogadásra kerül, ugyanazokat az állapotokat veheti fel, mint a „trade”.

- *Offer Details*: Ez az oldal az ajánlattevő számára biztosítja az ajánlat szerkesztését, törlését, valamint az ajánlat állapotának módosítását, ugyanazokkal a feltételekkel, mint amelyeket a *Trade Details* pontban ismertettem. A csere létrehozója szintén meg tudja tekinteni az ajánlat részletes oldalát, ahol látja az ajánlattevő elérhetőségeit, és lehetősége van az ajánlat elfogadására vagy elutasítására.
- *My Collection*: Ez az oldal a felhasználó gyűjteményét jeleníti meg. Itt lehetőség van a gyűjtemény adatainak szerkesztésére, például a név és a borítókép módosítására. Az oldal kilistázza a gyűjteményben található elemeket, és egy elem kiválasztásával a program annak részletes nézetéhez navigál.
- *Item Details*: Ez az oldal egy kollekcióelem részletes nézetét jeleníti meg, ahol minden elérhető információ pontosan megtekinthető róla. Itt van lehetőség az elem törlésére a kollekcióból, illetve adatainak módosítására. Gyakori használati eset például, hogy a felhasználó módosítja az elem darabszámát, ezzel jelezve, hogy a tárgyból fizikálisan több (vagy kevesebb) van.
- *Other User*: Ez az oldal nem az aktuális felhasználó adatait jeleníti meg, hanem egy másik felhasználóét, akire például a *Feed* vagy a *Barter* oldalról navigálhatunk el. Az oldalon részletes információk találhatók az adott felhasználóról, például a nemzetisége, elérhetőségei, valamint a befejezett cserék után kapott értékelései. Ezen kívül megjelennek a kollekciói, abból a kollekciótípusból, amelyet az aktuális felhasználó kiválasztott kollekciójának típusa ad. A kollekciók kiválaszthatók és részletesen megtekinthetők. Ez az oldal átfogó képet ad az adott felhasználóról és gyűjteményeiről.
- *Other User’s Collection*: Ezen az oldalon egy másik felhasználó gyűjteményét tekinthetjük meg. Ha megtetszik a gyűjtemény, feliratkozhatunk rá, így az megjelenik a *My Follows* oldalon. A

gyűjtemény elemeire kattintva a részletes információkat is megtekinthetjük.

- *Other User's Item Details*: Ezen az oldalon egy másik felhasználó kiválasztott kollekcióelemének részleteit tekinthetjük meg. Csakúgy, mint a *Feed* oldalon, itt is lehetőség van a tárgy kedvelésére és a ritkaságértékelésére. A ritkaságértékelés fokozatait a *Feed* pontban ismertettem.
- *Settings*: A Beállítások menüt a fejlécben található fogaskerék ikonnal tudjuk megnyitni. Itt információkat találunk saját felhasználónkról, valamint további aloldalak is elérhetők a különböző beállítási csoportokhoz. Például a Felhasználó beállítások oldalon tudjuk megadni elérhetőségeinket.
- *New Items*: A kollekció kiválasztása után a kontextusfüggő menükben a jobb alsó sarokban található plusz gomb segítségével tudunk új tárgyakat felvenni. Először ki kell választani a feltöltendő képet, fotózással vagy a galériából. Ezután a rendszer kiértékeli a képet, meghatározza az esetleges tárgyakat rajta, és kivágott képként jeleníti meg őket. Minden tárgyról a rendszer ad egy predikciót, hogy szerinte már létezik-e az adott elem a felhasználó kollekciójában, vagy új elemről van szó. Mivel a hasonlóságvizsgáló modell nem tökéletes, mindig lehetőség van a predikció felülbírálására. A felhasználó eldöntheti, hogy a modell által javasolt találat valóban létező elem, vagy új. Ha a modell több lehetséges találatot ad (például a három legvalószínűbbet), ezekből is lehet választani. Egy képen több tárgy is szerepelhet, ezért mindegyiket meg kell vizsgálni. Új tárgy esetén megadhatjuk az adatait, létező elem esetén pedig +1 értéket adhatunk hozzá a meglévő elemhez. A találatokat a pipa gombbal lehet elfogadni, az X gombbal pedig elutasítani; elutasítás esetén az adott elemhez nem történik változtatás, vagyis nem kerül fel a kollekcióba, illetve nem növekszik a mennyisége, ha már bent van. A műveletet a jobb felső sarokban található *Finish* gombbal lehet befejezni.

2 Alkalmazott technológiák

2.1 PostgreSQL

Minden alkalmazás alapját egy adatbázis képezi, ahova a felhasználók által generált adatok menthetők. Az adatbázisoknak számos követelménynek kell megfelelniük: biztosítaniuk kell az adatbiztonságot és az adatvédelmet, hibamentesen kell működniük, valamint támogatniuk kell a többfelhasználós környezetet.

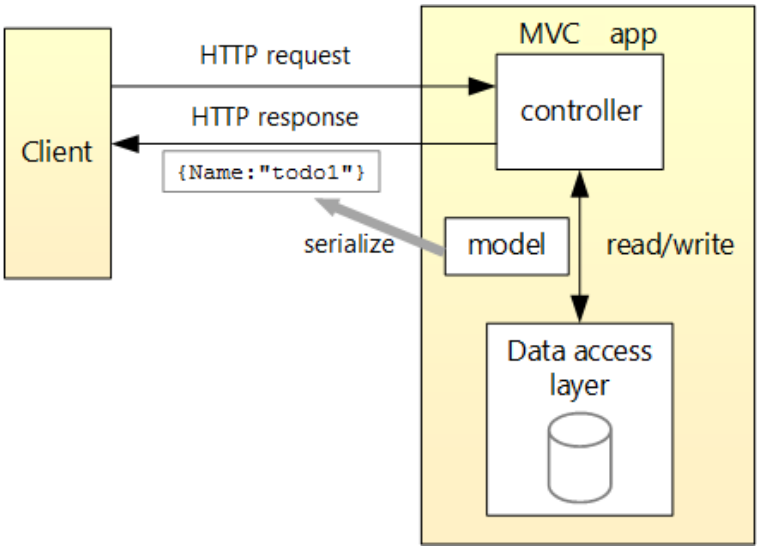
Napjainkban rengeteg lehetőség közül választhatunk - léteznek relációs és NoSQL adatbázisok is. Bár a NoSQL megoldások egyre nagyobb népszerűségnek örvendenek, én a relációs adatbázis mellett döntöttem, mivel főként ehhez tanultam meg az adatstruktúra-tervezés alapjait. A relációs adatbázisok nagy része hasonló elven működik, csak kisebb különbségek figyelhetők meg köztük - például bizonyos SQL-funkciók elérhetőségében. A választást általában nem ezek a különbségek, hanem sokkal inkább a fejlesztői tapasztalat, a használt eszközök ismerete, illetve a projektbe illeszthetőség határozza meg. Én is ezen a szemléleten haladtam, és azt a megoldást választottam, amellyel korábban már dolgoztam: így az adatbázisom PostgreSQL alapokon fut.

2.2 ASP.NET Core (backend)

Az applikáció backend szerverét ASP.NET Core technológiával valósítottam meg. Azért választottam ezt a keretrendszert, mert a C# nyelvre épül, amelyet már ismerek, és egyetemi tanulmányaim során is foglalkoztam az ASP.NET Core-ral - habár nem ilyen mélységben, mint ennél a projektnél. A döntésben az is szerepet játszott, hogy ez az egyik leggyakrabban használt technológia weboldalak és alkalmazások backend szervereként, amihez számos segédanyag és dokumentáció érhető el, így elakadás esetén gyorsan találhatók megoldások. A Microsoft által fejlesztett ASP.NET keretrendszer-család frontend- és backend-fejlesztéshez egyaránt kínál megoldásokat. Én ebből a családból a Core verziót használtam, amely több korábbi ASP.NET keretrendszer előnyeit egyesíti egy modern, egységes keretrendszerben. Az ASP.NET Core platformfüggetlen, nagy teljesítményű, és számos hasznos szerveroldali funkciót kínál. Az ASP.NET Core további előnyeiről a Microsoft hivatalos honlapján olvashatunk. [1]

2.2.1 Web API

A keretrendszer sok hasznos komponense közül a projekt során elsősorban az API-fejlesztési lehetőségeit használtam ki, mivel minimális konfigurációval is gyorsan létrehozható egy jól működő szerveroldali API. A frontend mobilalkalmazás ezen az API-n keresztül éri el az adatbázis adatait CRUD műveletekkel, valamint a bonyolultabb use case-ekhez külön, célzott végpontokat is kialakítottam.



2. ábra Tipikus ASP.NET Core Web Api design [12]

Egy ASP.NET Core Web API-val működő backend tipikus felépítését az 2. ábra szemlélteti. Az API-végpontok kontrollerekbe szerveződnek, amelyek az alkalmazás elérési rétegén keresztül kommunikálnak az adatbázissal. A kontrollerek felelősek a HTTP-kérések fogadásáért és kezeléséért, ahol az adatok JSON formátumban közlekednek. A JSON-csomagokból a JSON serializer a névkonvenciókat követve automatikusan képes modellobjektumokat létrehozni. A névkonvenciók jelentőségét a 2.3.1-es pontban ismertetem egy másik technológia kapcsán. A kontrollereket célszerű valamilyen logikai egység szerint elkülöníteni, például minden modellhez (egyszerű vagy összetett) egy külön kontroller tartozhat. A kontrollereken belül definiálhatók a végpontok, amelyek lehetnek egyszerű CRUD műveletek, vagy bonyolultabb üzleti logikát megvalósító hívások. A Web API előszeretettel használja a C# attribútumait a kontrollerek és végpontok implementálásához. További részletek és példák az attribútumokról, a Microsoft hivatalos C# dokumentációjában találhatók. [2]

```
[ApiController]
[Route("api/[controller]")]
public class BookController : ControllerBase
```

3. ábra Kontroller osztály fejléc

Kontroller osztály létrehozásához a 3. ábra szerint kell eljárni. Minden kontroller osztályt először származtatunk a *ControllerBase* ősosztályból, valamint ellátjuk az *ApiController* attribútummal. A *Route* attribútum adja meg az alap URL-jét a kontrollernek, vagyis azt, hogy az itt található végpontok URL-je hogy fog kezdődni.

```
[HttpGet("{id}")]
public virtual async Task<ActionResult<Book>> GetById(Guid id)
{
    var item = await _repository.GetItemByIdAsync(id);
    if (item == null) return NotFound();
    return Ok(item);
}
```

4. ábra Egyszerű Book objektum lekérdezése id alapján

A kontrollerosztály létrehozása után elkezdhetjük feltölteni az osztályt végpontokkal, amelyek lényegében aszinkron függvények lesznek. Minden végpontfüggvény definíciója elé meg kell adnunk egy attribútumfejlécet, amely meghatározza, milyen típusú API-végpontot hozunk létre. Amennyiben a végpont az URL-ben paramétert is vár, azt szintén itt szükséges megadni. A 4. ábra egy GET típusú végpontot mutat, amely egy paramétert, egy könyv azonosítóját várja az URL-ben. A paraméter természetesen a függvény paraméterlistájában is megjelenik. Minden végpontnál meg kell adni a megfelelő attribútumot, hogy a rendszer tudja, milyen típusú hívásról van szó.

Web API végpont attribútumok:

- *[HttpGet]*: lekérdezéshez
- *[HttpPost]*: létrehozáshoz
- *[HttpPut]*: módosításhoz
- *[HttpDelete]*: törléshez

A függvény paramétereinek is adhatunk attribútumokat. A *[FromQuery]* attribútum azt jelzi, hogy az adott paraméter az URL keresési paraméterei közül származik, míg a *[FromBody]* attribútum esetén az adat a HTTP-kérés törzséből érkezik. Az API-függvények visszatérési értéke mindig valamilyen HTTP-választípus, amely

lehet üres, vagy tartalmazhat adatot is. Emiatt a visszatérési értéknek általában *ActionResult* típusúnak kell lennie, mivel a konkrét HTTP-választípusok ebből származnak le.

2.2.2 Dependency Injection

Alkalmazásfejlesztés során mindig fontos szempont, hogy a rendszer bármely komponense később lecserélhető legyen más implementációra. Ezt interfészek használatával lehet megvalósítani, azonban minden interfészhez meg kell adnunk egy implementációt is, ami nagyobb projektekben bonyolulttá válhat, ha a függőségeket nem kezeljük egységesen. Ezért a fejlesztés során tudatosan nem csatoltam közvetlenül konkrét implementációkat a komponensekhez, hanem csak interfészeket használtam, és a függőségek kezelésére függőség-injektálást alkalmaztam. Az ASP.NET Core egyik nagy előnye, hogy ehhez beépített, teljes körű támogatást biztosít, ami jelentősen megkönnyíti a fejlesztést és a kód karbantarthatóságát. Függőséginjektálásnál célszerű egy közös fájlt létrehozni, ahol ezt elvégezzük.

```
services.AddScoped<IBookRepository, BookRepository>();
services.AddTransient<IBookRepository, BookRepository>();
services.AddSingleton<ITempTokenService, TempTokenService>();
```

5. ábra A három leggyakoribb függőséginjektálás

A függőséginjektálást a keretrendszer által biztosított *IServiceCollection* objektumon keresztül végezzük. Ez az objektum felel azért, hogy a program automatikusan kiossza a megfelelő implementációkat azokon a helyeken, ahol az adott interfész vagy szolgáltatás szükséges.

A függőségek regisztrálása élettartam (lifetime) alapján többféleképpen történhet:

- *AddScoped()*: A szolgáltatás példánya a hívási kontextus idejére (például egy webkérés teljes időtartamára) jön létre, majd a kontextus lezárultával megszűnik. Gyakran használjuk repositoryk regisztrálásakor.
- *AddTransient()*: Minden alkalommal új példány jön létre, amikor az adott szolgáltatásra szükség van. Ez akkor hasznos, ha az implementáció nem tart fenn állapotot.
- *AddSingleton()*: A szolgáltatás példánya a program indulásakor jön létre, és a teljes alkalmazás futása során ugyanaz a példány kerül felhasználásra.

Ezt akkor célszerű alkalmazni, ha több komponensnek ugyanazt az állapotot vagy erőforrást kell megosztania.

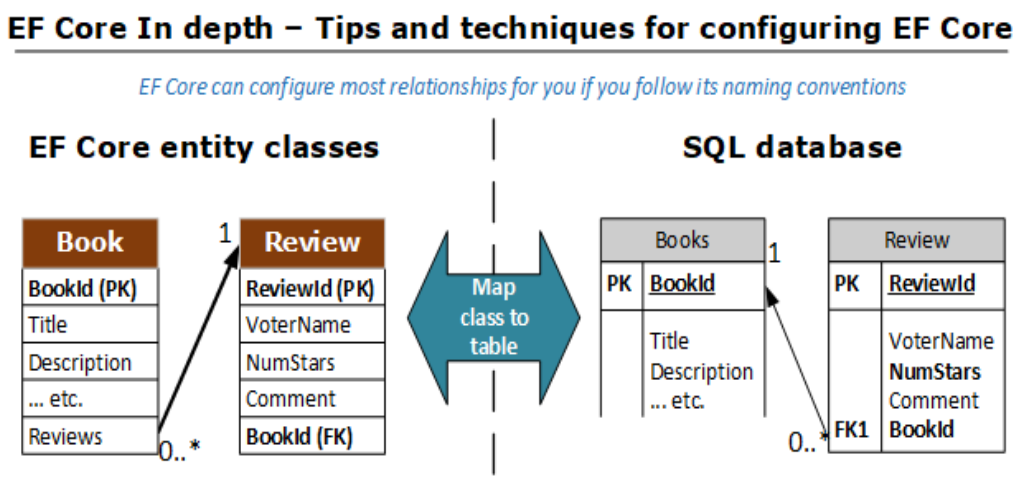
Mindezeken kívül számos további beépített keretrendszer-funkció is regisztrálható függőséginjektálással, például az adatbáziskontextus vagy az API-autorizáció.

2.3 Entity Framework

A Microsoft egyik jelentős fejlesztése az Entity Framework Core, amely eredetileg Entity Framework néven indult, és kezdetben kizárólag objektum-relációs leképezést (ORM) biztosított. Az új, továbbfejlesztett verzió - az Entity Framework Core - ezt a funkcionalitást számos új képességgel bővítette, például a platformfüggetlenséggel. Bár ez a keretrendszer nem része közvetlenül az ASP.NET Core-nak, a kettőt gyakran együtt használják. Én is mindkettőt alkalmaztam a projekt során. Számomra az Entity Framework Core legfontosabb funkciói az objektum-relációs leképezés, valamint a navigációs tulajdonságok voltak. A navigációs tulajdonságok használatát a Microsoft hivatalos dokumentációja szemlélteti példákon keresztül. [3]

2.3.1 Objektum-relációs leképezés

Az objektum-relációs leképezést (ORM) megvalósító könyvtárak automatikusan elvégzik a konverziót a relációs adatbázis táblái és az objektum-orientált programozási nyelv osztályai között. Ez azt jelenti, hogy ha ilyen könyvtárat használunk, nem kell külön logikát írnunk ahhoz, hogy a táblákból érkező rekordokat a megfelelő objektumokhoz rendeljük, mert ezt a könyvtár automatikusan kezeli. A megfeleltetés fordított irányban is működik: adatíráskor az objektum alapján a neki megfelelő rekord kerül be az adatbázis táblájába.



6. ábra Példa objektum-relációs leképezésre [13]

A leképezés lényegét az 6. ábra példáján keresztül szeretném bemutatni. Az SQL-adatbázisban két tábla található: *Books* és *Review*. Ehhez létrehozunk két C# osztályt, amelyeket tetszőlegesen elnevezhetünk. A lényeg az, hogy az osztály tulajdonságai az aktuális névkonvenciót kövessék. A névkonvenció határozza meg, hogy az adatbázis-tábla mezőihez mely C# osztálytulajdonságok rendelődnek hozzá. A példában a *PascalCase* konvenció van érvényben, így a könyvtár tudja, hogy a *BookId* C# tulajdonság a tábla *BookId* mezőjéhez tartozik. Minden ORM-könyvtár rendelkezik egy alapértelmezett névkonvencióval, amely szükség esetén módosítható.

A név konvenciós szabályok:

- *Camel Case*: firstName
- *Snake Case*: first_name
- *Kebab Case*: first-name
- *Pascal Case*: FirstName

```
public DbSet<Book> Books { get; set; }
```

7. ábra Táblázat reprezentáció Entity Framework Core-ban

A táblázat nevénél is figyelni kell a névkonvencióra. A táblázat Entity Framework Core-ban, *DbSet*-ként van reprezentálva, itt a változó neve a névkonvenció alapján meg kell egyezzen a táblázat nevével.

2.3.2 Navigációs tulajdonságok

A navigációs tulajdonságok rendkívül hasznosak: segítségükkel az adatbázisban valamilyen asszociációban álló táblák között, a C#-ban reprezentált objektumok szintjén könnyedén navigálhatunk, anélkül hogy a kapcsolótáblát külön le kellene kérdeznünk és manuálisan összekapcsolnunk az adatait. Ez azt jelenti, hogy egy osztályban tulajdonságokkal jelölhetjük a táblák közötti kapcsolatokat, így a kapcsolat másik oldalán lévő C#-objektumok bármely elemét közvetlenül elérhetjük. A 2.3.1-es pontban ismertetett példánál maradva: ez lehetővé teszi például, hogy egy könyv objektumon keresztül közvetlenül elérjük a hozzá tartozó értékeléseket, vagy fordítva, egy értékelésből visszahivatkozzunk a kapcsolódó könyvre.

```
//Review osztály tulajdonság
public Book Book { get; set; }
// Book osztály tulajdonság
public ICollection<Review> Reviews { get; set; }
```

8. ábra Navigációs tulajdonságok a másik osztályra hivatkozva

A 8. ábra a navigációs tulajdonságok C# kódját mutatja. Amikor pontosan egy másik objektumra hivatkozunk, az egy egyszerű tulajdonság a hivatkozott objektum típusával, amikor több másik objektumra hivatkozunk, akkor az *ICollection*-ként kell felvenni, ahol az *ICollection* típusa a másik objektumok típusa. Itt nincsen névkonvenció, mert ez nem egy automatikus objektum-relációs lekérdezés megoldás.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    //Book és Review navigációs tulajdonság összekötés
    modelBuilder.Entity<Book>()
    .HasMany(b => c.Reviews)
    .WithOne(r => r.Book)
    .HasForeignKey(r => r.BookId);
}
```

9. ábra Kapcsolat meghatározása a navigációs tulajdonságokknak

Ahhoz, hogy a navigációs tulajdonságok megfelelően működjenek, az 9. ábra szerint kell eljárnunk. Az *OnModelCreating()* függvény minden esetben automatikusan lefut, amikor a rendszer a program indításakor az adatbázis alapján létrehozza a modelleket és a közöttük lévő kapcsolatokat. A navigációs tulajdonságokat ebben a metódusban tudjuk definiálni. Az ábrán látható, hogy megadjuk az asszociáció típusát -

jelen példában ez egy egy-több kapcsolat. Itt határozzuk meg, hogy az osztályokban létrehozott *Reviews* és *Book* tulajdonságok navigációs tulajdonságként működnek, valamint hogy a *Review* osztály *BookId* tulajdonsága idegen kulcs. Fontos megemlíteni az *Include()* függvény szerepét is. Bár ennek használata konfiguráció kérdése, alapértelmezés szerint az Entity Framework nem tölti be automatikusan a navigációs tulajdonságokat az adatbázisból. Ez problémát okozhat, ha egy olyan objektumra hivatkozunk, amely csak a navigációs tulajdonságban érhető el, mert ilyenkor annak értéke *null* lesz, és a program futási hibát eredményezhet. Ennek elkerülése érdekében a szülő objektum lekérdezésekor meg kell hívni az *Include()* függvényt, amely gondoskodik arról, hogy a navigációs tulajdonság értékei is betöltődjenek az adatbázisból, így azokat a programban biztonságosan használhatjuk.

2.4 React Native

A frontend keretrendszerek közül én a React Native-et választottam. Azért esett erre a választásom, mert Reactben már sok tapasztalatot szereztem, így a React Native elsőre is nagyon ismerősnek tűnt. Később, jobban megismerve is azt tapasztaltam, hogy nagyban hasonlít a Reactre, a legnagyobb különbséget talán a beépített JSX tagek jelentették. A megszokott könyvtáraimat is tudtam használni, hiszen ami Reactben működik, az általában React Native-ben is működik, a frontend-komponenskönyvtárakat leszámítva. További ok volt a választás mellett, hogy korábban már volt tapasztalatom az Android Studio mobilfejlesztési környezetével is. Ugyanakkor azt tapasztaltam, hogy az Android Studio fejlesztés közben nagyon erőforrás-igényes. Mivel Reacttel gyorsan lehet fejleszteni, azt gondoltam, hogy a React Native esetében is így lesz - azonban tévedtem: ez is megterheli a számítógépet, de nem annyira, mint az Android Studio.

2.4.1 JSX Tagek

Ahogy már említettem, a React Native-ben használt tagek nem egyeznek meg a Reactben használt HTML-tagekkel. Itt más típusú tagek találhatók, és mivel a React Native platformfüggetlen, ezek minden platformon a saját natív kódjuk segítségével vannak implementálva.

```
<View>
  <Text>Hello World!</Text>
  <Image source={require("./random.png")} />
  <TouchableOpacity>Click me!</TouchableOpacity>
</View>
```

10. ábra Gyakori React Native JSX tagek

A 10. ábra a React Native-ben használt leggyakoribb JSX-tageket mutatja be. A <*View*> nagyjából megfeleltethető a HTML-ben használt <*div*> tagnak, a <*Text*> a <*p*> tagnak, az <*Image*> pedig az <*img*> tagnak. A <*TouchableOpacity*> viszont már kifejezetten a React Native saját tage, amely egy olyan gombot jelöl, ami érintésre elhalványul.

2.4.2 Expo

A React Native egy kiváló eszköz platformfüggetlen frontend kód fejlesztésére, azonban önmagában csak erre alkalmas. Mi, szoftverfejlesztők viszont tudjuk, hogy egy szoftver fejlesztése sokkal többről szól, mint pusztán a kódolásról. Ide tartozik például a build folyamat szervezése vagy a programkönyvtárak menedzselése is. A React Native önmagában ezekben a területekben hiányosságokat mutat. Erre kínál megoldást az Expo, amely egy jól kidolgozott, számos hasznos funkciót tartalmazó eszközkészletet biztosít a React Native köré. A fejlesztés során én is ezt az eszközkészletet használtam a buildek készítésére, a könyvtárak menedzselésére, valamint az Android metaadatok szerkesztésére. További információk az Expo hivatalos dokumentációjában találhatók. [4]

2.4.3 React Native Elements

Frontend-fejlesztés során a logika és a könnyű használhatóság mellett fontos szerepet kap a dizájn is. Gyakran előfordul, hogy a képernyőre nem elég egy egyszerű gomb – több kell ahhoz, hogy a felhasználói élmény egyértelmű és teljes legyen. Az évek során a frontend-fejlesztésben kialakultak olyan komponensek, amelyeket mind a fejlesztők, mind a felhasználók elfogadtak, mert jól használhatók, és a felhasználók már első pillantásra tudják, hogyan működnek. A fejlesztők, hogy kövessék ezeket a konvenciókat, sok ilyen komponenst nem maguk készítenek el, hanem valamilyen UI-eszközkészletet használnak, amelyben ezek a komponensek már előre meg vannak írva,

és csak illeszteni kell őket az alkalmazás dizájnjához. Én kész komponenseket a React Native Elements [5] UI-eszközkészletből vettem. Ez az eszközkészlet számos hasznos komponenst ad hozzá a React Native-hez, például *<BottomSheet>*, *<Avatar>* vagy *<FAB>* (floating action button) komponenseket, amelyek mind nagyon gyakoriak a mobilalkalmazásokban. Ha a feladatom bemutatásánál olyan JSX tag van, ami nem ismerős a React Native-ből, akkor az vagy egy saját komponens, vagy a React Native Elements-ből származik.

2.5 Python Service FastAPI

A projektben előre megjegyzem, hogy az AI-val kapcsolatos részek Pythonban íródtak, és az ASP.NET Core backend mellett egy Python service is aktív, amely másik backendként működik. Az ASP.NET Core backend ehhez a Python service-hez fordul, amikor AI-hívás történik. A két backend közötti kapcsolat szintén API-hívásokkal van megoldva. A Python service API-végpontjai a FastAPI Python könyvtár segítségével működnek.

```
@app.get("/book/{id}")
async def get(
    id: str
):
    return {"book" : "book1" }
```

11. ábra Fast Api GET végpont

A FastAPI-végpontokat hasonlóan egyszerű létrehozni, mint a Web API-végpontokat. Ez egy könnyebb, de rendkívül gyors API-könyvtár, így konfigurációs lehetőségekből kevesebb áll rendelkezésre. Miután a fejlesztő inicializálja a FastAPI-t a *FastAPI()* konstruktorral, már azonnal létrehozhatók a végpontok. Egy végponthoz az azt megvalósító függvény fölé egy @ dekorátor szimbólumot kell elhelyezni, amelyben meg kell adni, milyen típusú végpontot hozunk létre. Hasonlóan a 2.2.1-es pontban ismertetett Web API-hoz, itt is a dekorátor paramétereként, valamint a függvény paraméterlistájában megadva lehet az URL-ben paramétert továbbítani.

2.6 Tárgyfelismerés

A tárgyfelismerés, vagy más néven objektumdetektálás, a programom egyik legfontosabb része. Ez teszi lehetővé, hogy a felhasználó által készített képből a rendszer

kivágja azt a részt, amely a kollekcióba felvenni kívánt tárgyat tartalmazza. Mivel a legtöbb könyvtár és segédanyag mesterséges intelligenciához Python nyelven érhető el, ezért a tárgyfelismerést Pythonban valósítottam meg. Objektumdetektálásra nem létezik univerzális megoldás. Ahhoz, hogy a detektáló mesterséges intelligencia pontosan és megbízhatóan működjön, be kell tanítani egy modellt az adott objektumtípus felismerésére. A betanításhoz azonban - a felismerendő objektum komplexitásától függően - nagyszámú, felcímkézett tanítóadat szükséges. Mivel a projektemhez nem találtam megfelelő, felcímkézett tanítóadatokat, ezért ezeket magam hoztam létre a Label Studio segítségével.

2.6.1 Label Studio

A Label Studio egy nyílt forráskódú adatcímkéző alkalmazás. Nem önálló program, mivel a futtatásához Python környezet szükséges, és egy Python-szkript segítségével telepíthető. Az alkalmazás rendkívül hasznos, mivel egy egyszerűen használható, grafikus felhasználói felületet biztosít, amelynek segítségével gyorsan felcímkézhetők az adatok - legyen szó képfeldolgozásról, hangfeldolgozásról, természetes nyelvfeldolgozásról vagy videófeldolgozásról. A Label Studio hivatalos dokumentációja tartalmaz egy sablonoldalt, amely bemutatja az összes elérhető címkézési funkciót [6]. A felcímkézési folyamat bármikor elmenthető, és az alkalmazás számos importálási és exportálási lehetőséget támogat. A dokumentáció [7] több exportformátumot is felsorol, ezek közül én a YOLO formátum 8-as verzióját használtam, mivel ez kompatibilis a betanítási modell architektúrájával.

2.6.2 YOLOv8

A számítógépes látás egyik uttörő model architectúrája a YOLOv8. Számos alkalmazás használja detektáláshoz, osztályozáshoz. Használata betanításhoz nagyon egyszerű.

```python
# Előre betanított alapmodell betöltése
model = YOLO("yolov8n.pt")

# Betanítás saját adathalmazon
model.train(
    data="datasets/dataset/data.yaml",
    epochs=50,
    imgsz=640,
    batch=16
)
```

12. ábra YOLOv8 modell betanítása

A 12. ábra a betanítás menetét mutatja. Először be kell tölteni egy alapmodellt, majd elindíthatjuk a tanítást a saját adathalmazunkon. Az adathalmaz elérési útja mellett meg kell adni az *epochs* értékét, vagyis azt, hogy a betanítás hányszor menjen végig a teljes adathalmazon, hogy a modell módosíthassa a súlyait. Ez az érték lényegesen meghatározza a tanítás idejét: ha elegendő adat áll rendelkezésre, érdemes magasabb értéket választani, mert pontosabb modellt eredményezhet. Ugyanakkor kis adathalmaz esetén kerülni kell a túl magas értéket, mert a modell túltanulhat (overfitting léphet fel). Harmadik paraméterként meg kell adni azt a képméretet, amelyen a rendszer tanulni fog. A YOLOv8 minden képet erre a méretre skáláz át, így az eredeti képek mérete nem kell, hogy egyezzen ezzel az értékkel. Végül meg kell adni a batch méretet, amelynek nem kötelező 2 hatványának lennie, de gyakran praktikus úgy megválasztani (például 8, 16, 32 stb.). A batch méret azt határozza meg, hogy a rendszer egyszerre hány képet dolgoz fel. Alacsony érték esetén a tanítás lassabb, de kevesebb GPU-memóriát igényel; nagyobb érték gyorsabb tanítást eredményez, viszont több memóriát igényel. Ahogy a példa is mutatja, egy ilyen modell betanítása viszonylag egyszerű, és a paraméterekkel való kísérletezés is könnyen elvégezhető, mivel kevés, jól érthető beállításról van szó. Egy dologra azonban különösen figyelni kell: a saját adathalmaznak a YOLOv8 által elvárt mappastruktúrában kell lennie, hogy a rendszer megfelelően fel tudja dolgozni a tanításhoz.

Az mappastruktúra felépítése:

dataset mappa(bármi lehet a neve)
images mappa
train mappa
val mappa

labels mappa
train mappa
val mappa

A *train* mappák tartalmazzák a betanításhoz használt képeket és a hozzájuk tartozó címkéket, míg a *val* mappák az ellenőrző (validációs) adatokat. A modell a betanítás során ezeket az adatokat használja önellenőrzésre, vagyis ezek segítségével méri fel, mennyire jól tanult, mivel ezek nem részei a tanító halmaznak. A betanítás befejeztével a rendszer létrehoz egy *runs* mappát, amelybe elmenti a metrikákat (például pontosság, veszteség) és a betanított modellt is, jellemzően *best.pt* néven.

```
# YOLO modell futtatása képen
model = YOLO("runs/detect/train/weights/best.pt")
results = model.predict("testdata/cap2.jpg", imgsz=512, conf=0.7)
```

13. ábra Betanított YOLO modell használata

A betanított YOLOv8 modell használatát a 13. ábra szemlélteti. Először be kell tölteni a betanított modellt, majd meg kell hívni a *predict()* függvényt. A függvény paramétereként meg kell adni a vizsgált képet, a képméretet, amelyen a vizsgálatot futtatja, valamint egy határértéket, amely fölött az objektumot felismeri. A visszatérési érték egy tömb, mivel egyszerre több képet is lehet vizsgálni. Minden képhez tartozik egy összetett objektum, amely számos eredményadatot tartalmaz.

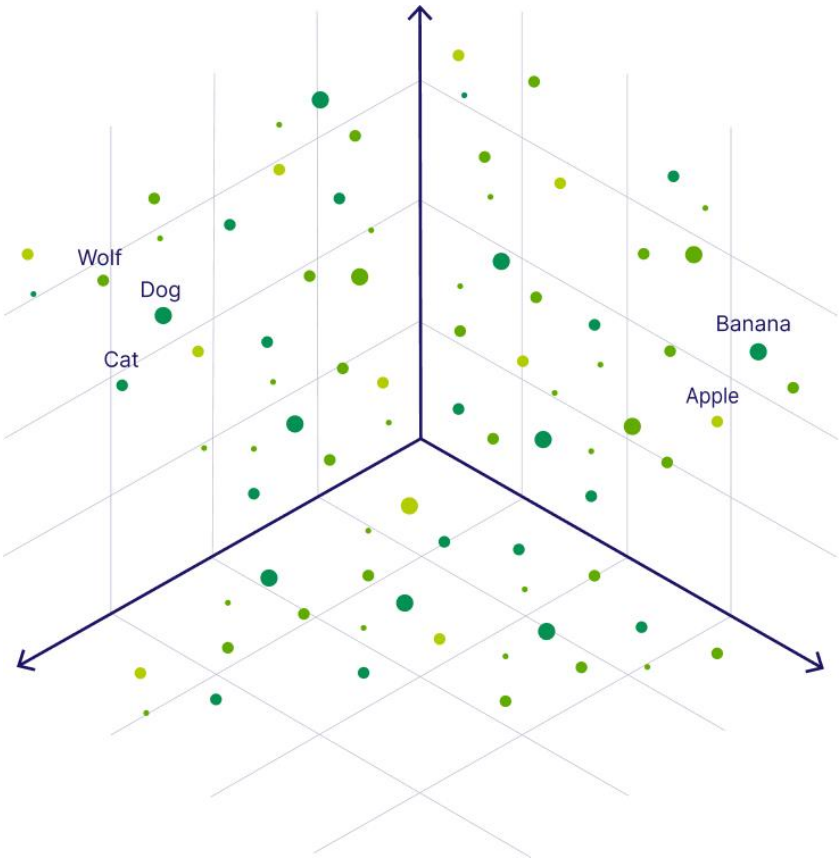
2.7 Hasonlóságvizsgálat

Az AI-t a világunkban legtöbbször összehasonlításra használjuk: két vagy több dolgot hasonlítunk össze a segítségével - legyen szó tárgyakról vagy akár nem kézzel fogható fogalmakról, mint képek hasonlóságáról, hangok azonosításáról, szövegek összevetéséről vagy akár személyes ajánlórendszerekről, amelyek felhasználói preferenciákat hasonlítanak össze). Az összehasonlítás valamilyen formában szinte mindenhol jelen van, még olyan helyzetekben is, ahol elsőre nem is gondolnánk rá - például amikor egy önvezető autó különbséget tesz gyalogos és közlekedési tábla között,

egy keresőmotor rangsorolja a találatokat hasonlóság alapján, vagy egy fordító AI több nyelvi jelentés közül választja ki a legmegfelelőbbet. Egy AI-modell használata összehasonlításra azért különösen hasznos, mert egy jól megtervezett modell nagyon hatékonyan működik, és megfelelő tanítóadat-halmaz esetén viszonylag gyorsan tanítható. Sok esetben a problémára algoritmust írni jóval időigényesebb, hibalehetőségekkel teli, erőforrás-igényes, vagy egy komplex összehasonlítás esetén akár szinte lehetetlen feladat is lehet. A saját projektemben én is alkalmazok összehasonlítást: annak meghatározására, hogy egy kollekcióba kerülő tárgy már létezik-e a gyűjteményben a fényképe alapján, vagy sem.

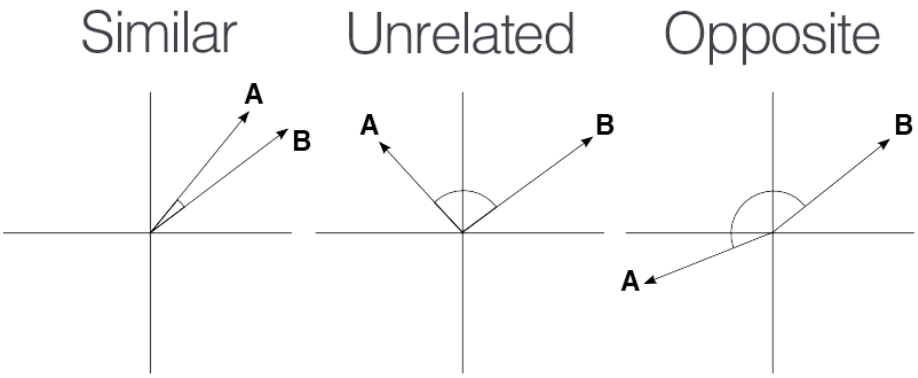
2.7.1 Embedding

Amikor először kezdtem el foglalkozni a mesterséges intelligenciával, leginkább az foglalkoztatott, hogyan képes egy AI megérteni egy szöveget vagy felismerni egy képet - majd ezek alapján értelmes választ adni, illetve pontosan leírni, mi látható a képen. Vagyis hogyan tudja az AI elvégezni mindazt, ami nem előre meghatározott szabályokra épül, hanem amit az ember a saját tapasztalata és intelligenciája alapján ismer fel. A válasz az embedding, vagyis az a folyamat, amikor a szövegeket, képeket vagy akár videókat egy többdimenziós vektortérben ábrázoljuk. A mesterséges intelligenciának ugyanis jelenleg sincs „agyi” értelemben vett intelligenciája - továbbra is a számítógép számítási teljesítményére támaszkodik, tehát számokkal dolgozik. Ezért szükséges, hogy a nagyon komplex struktúrákat, mint például a szöveget vagy a képet, numerikus formába, sok dimenzióból álló vektorokká alakítsuk át. Ebben a vektortérben az AI a „gondolatait” - vagyis a tanult mintákat és jelentéseket - képes kifejezni számok formájában. Ahogyan azt az IBM blogjában [8] is olvashatjuk, az embedding során az így létrehozott sokdimenziós vektorokat úgy helyezzük el a vektortérben, hogy azok szemantikusan értelmezhetők legyenek más AI-algoritmusok számára is. A beágyazást (embeddinget) egy modell végzi, amely általánosabb, egyszerű szemantikai problémák esetén nagyon jól működik. Ilyen például, amikor két macskát ábrázoló kép közel kerül egymáshoz a vektortérben, így a rendszer mindkettőről „tudja”, hogy macska látható rajtuk. Összetettebb problémák esetén azonban az általános modelleket tovább lehet tanítani vagy finomhangolni, hogy pontosabban működjenek egy adott, bonyolultabb feladaton.



14. ábra Embedding vektortér példa [9]

A 14. ábra egyszerű vektortér példát szemléltet. Ahogy a példában is látszik, azok a szavak, hogy „Wolf”, „Dog”, „Cat” közel vannak egymáshoz a vektortérben, mert mindegyik egy állatot reprezentál, valamint a „Banana”, és az „Apple” is közel vannak egymáshoz, mert mindkettő gyümölcs, viszont a két csoport messze van egymástól, mert teljesen két különálló fogalmat jelölnek.



15. ábra Cos hasonlóság vizsgálat vektorokon [10]

Az utolsó részben ismertetem, hogyan lehet két elem hasonlóságát mesterséges intelligencia segítségével megmérni. Első lépésként mindkét elemhez elkészítjük az

embedding vektorát. Ezt követően a két vektor közötti hasonlóság mértékét a koszinusz-hasonlóság (cosine similarity) segítségével határozhatjuk meg. Ha a két vektor kis szöget zár be egymással, az azt jelenti, hogy nagyjából azonos irányba mutatnak - vagyis a két embedding, és így a két vizsgált elem is hasonlónak tekinthető. Ezzel szemben, ha a két vektor nagyobb szöget zár be, az azt jelzi, hogy különböző irányba mutatnak, tehát a modell szerint a két elem jelentése eltérő.

2.7.2 Metric Learning

A projektben kevés tanító adat állt rendelkezésre, mivel a képek felcímkézése időigényes volt, és nehéz volt elegendő mennyiségű, különböző nézőpontból és fényviszonyok között készült képet találni ugyanarról a tárgyról (az én esetemben kupakról). Ezért választottam a Metric Learninget a modell fő tanulási elvének. Erre azért is szükség volt, mert a feladat nem osztályozási jellegű - tehát nem azt kell meghatározni, milyen típusú kupak látható a képen, hanem azt, hogy különbözik-e a többitől, vagyis hasonló-e egy adott példához. A Metric Learning pontosan ilyen problémákra alkalmas, mert nem osztálycímkéket tanul meg, hanem egy távolságmetrikát az embeddingek között, a hasonló képek vektorai közel kerülnek egymáshoz, a különbözőek vektorai pedig távol. Számos veszteség függvény alkalmazható Metric Learning [11] tanulásnál, én a tanulást a Triplet Loss és a Cross-Entropy Loss kombinációjával valósítottam meg. Ehhez a tanító adatokat úgy szerveztem, hogy a gyökérmappa osztályokat tartalmaz, minden osztály egy külön mappa, amelyben ugyanarról a tárgyról több kép található, különböző nézőpontból, megvilágításból és minőségben.

A mappastruktúra felépítése:

dataset mappa(bármi lehet a neve)

class1 mappa(bármi lehet a neve)

class 2 mappa(bármi lehet a neve)

A Triplet Loss úgy próbál metrikát meghatározni, hogy minden tanítási példát hármasban kezel:

- Anchor(A): kiindúló minta
- Positive(P): ugyanahoz az osztályhoz tartozó hasonló minta
- Negative(N): más osztályhoz tartozó különböző minta

A cél:

$$\mathrm{d}(\mathrm{A}, \mathrm{P}) + \mathrm{margin} < \mathrm{d}(\mathrm{A}, \mathrm{N})$$

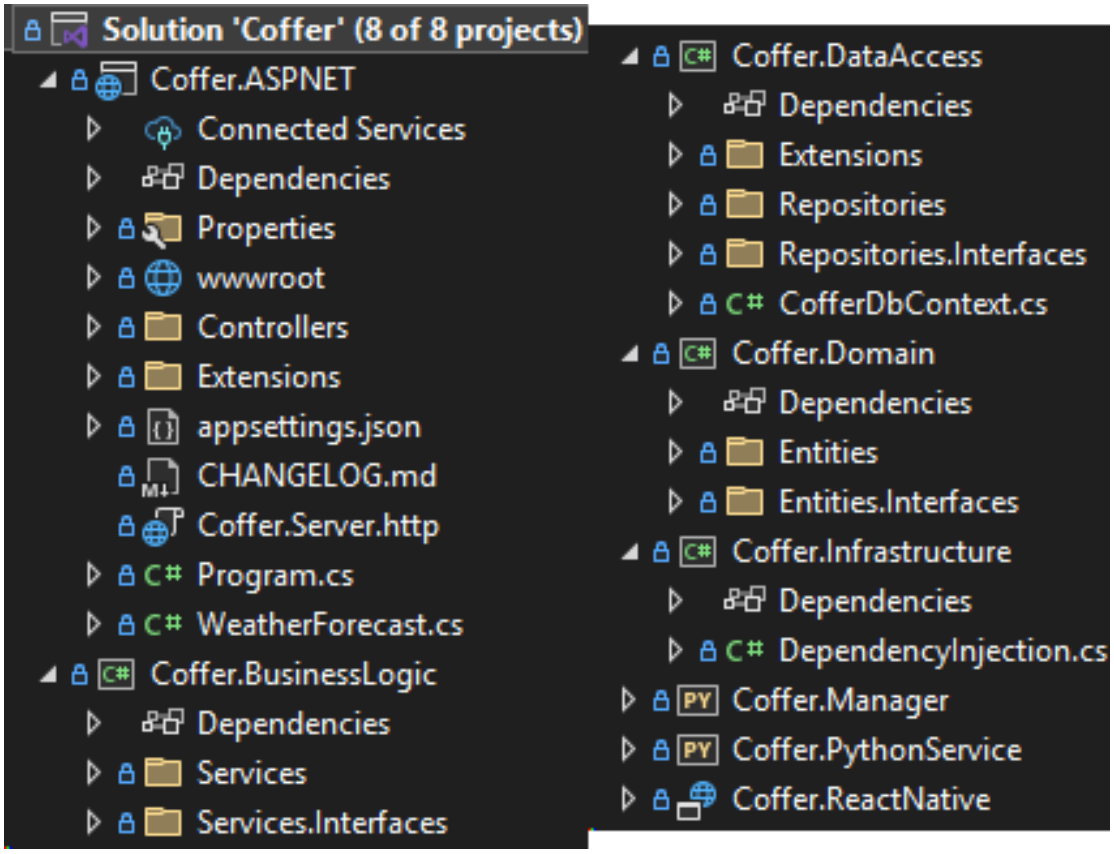
vagyis az anchor–positive távolság legyen kisebb, mint az anchor–negative távolság, egy bizonyos margin értékkel. Ezzel a modell megtanulja, hogy az azonos kategóriájú minták embeddingjei közel legyenek egymáshoz, a különbözők pedig távol.

A Cross-Entropy Loss egy egyszerűbb veszteségfüggvény, amely bünteti a helytelen osztályba sorolást, és jutalmazza a helyes besorolást a predikció valószínűségei alapján. Ha a Cross-Entropy-t és a Triplet Loss-t egyszerre alkalmazzuk, a modell egyszerre jó tud megkülönböztetni és hasonlóságérzékeny embeddingeket tanul meg, tehát jól tudja megkülönböztetni a különböző példákat, miközben a hasonló minták vektorai közel maradnak egymáshoz.

3 Architektúra

A szoftverfejlesztés során az architektúrára minden szinten figyelni kell, hiszen mindegyik szint valamilyen módon befolyásolja a fejlesztés hatékonyságát. Én két fő területen határoztam meg tudatos architekturális elveket. Az egyik az adatbázisséma szintje, amely a legfontosabb és egyben elengedhetetlen, hiszen közvetlen hatással van a program funkcionális megvalósíthatóságára, teljesítményére, valamint a fejlesztés nehézségére is. A másik architekturális szint, amelyre figyelmet fordítottam, a projekt mappastruktúrájának logikus kialakítása volt. Ez elsősorban nem a fejlesztés logikai bonyolultságát csökkenti, hanem a fejlesztés kényelmét növeli: meghatározza, milyen gyorsan találunk meg egy fájlt, hová érdemes újat létrehozni, és megkönnyíti más fejlesztők számára is a projekt megértését.

3.1 Mappastruktúra



16. ábra A projekt mappastruktúrája Visual Studio-ban

A projektet a különböző fő megvalósítási egységek mentén alprojektekké osztottam fel. A Visual Studio lehetőséget biztosít arra, hogy egy Solution-höz több

alprojektet is hozzáadjunk - ezt a lehetőséget használtam ki. Ennek eredményeként egyrészt kialakult a kívánt mappastruktúra, másrészt a 16. ábra által szemléltetett módon a Visual Studio is felismeri az egyes alprojekteket, és a használt fejlesztési nyelv alapján kiegészítő funkciókat kínál (például Python esetén különböző Python-környezetek létrehozását). Az *ASP.NET, BusinessLogic, DataAccess, Domain* és *Infrastructure* alprojektek logikailag a közös backendhez tartoznak, ugyanakkor így jól elkülöníthetők egymástól. A többi alprojekt a fő projekten belül teljesen önálló, külön is futtatható egységet alkot.

Alább az alprojektek rövid bemutatása következik:

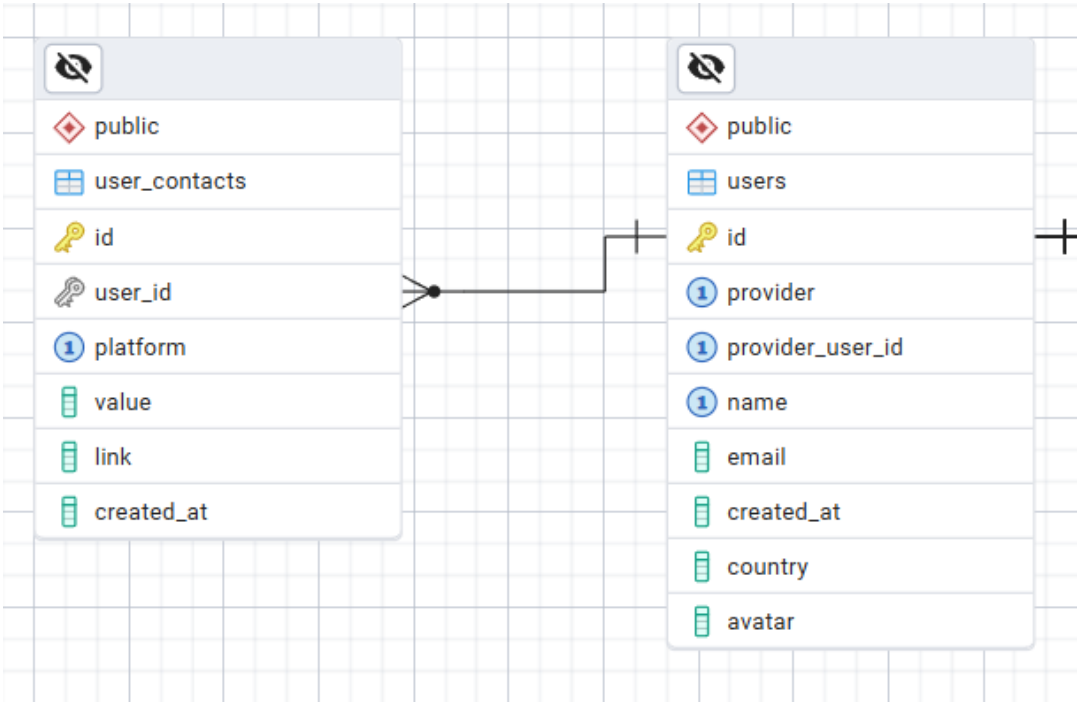
- *ASP.NET*: A backend projekt kiinduló pontja, a Visual Studio alapértelmezett beállítása szerint itt kapott helyet a *Program.cs*. Ezen kívül kiegészítőket tartalmaz vagy a *Program.cs*-hez, vagy a kontrollerekhez. Itt van definiálva az összes kontroller, amelyek kiszolgálják az API végpontokat.
- *BusinessLogic*: Itt vannak meghatározva a különböző szolgáltatások és azok interfészei. A szolgáltatások (service-ek) olyan részegységek, amelyek valamilyen összetettebb logikát valósítanak meg. Itt találhatók például a GitHub-belépésért felelős szolgáltatás, a JWT-tokenek kezelését végző szolgáltatás, valamint a képműveleteket megvalósító szolgáltatás.
- *DataAccess*: A Repository mintát követve itt találhatók a repository-k és azok interfészei. Ez az alprojekt felel az egyes osztályokhoz tartozó nyers adatok eléréséért az adatbázisból. A kiegészítők mappában *Include* függvények vannak, amelyek meghatározzák, hogy az adott osztályokból létrehozott objektumokhoz mely navigációs tulajdonságok adatai csatlakozzanak.
- *Domain*: Az entitásokat foglalja magába, amelyek az adatbázis sémákat modellezik C# osztályokkal.
- *Infrastructure*: A központi hely a függőséginjektálás számára. Jelenleg ez az egy funkció-ja van, de ha bonyolultabb, éles projekt lenne, ide jöhetnének a konfigurációs beállítások.

- *Manager*: Ez a Python projekt tartalmazza azokat a kódokat, amelyek a tárgydetektálás és a hasonlóságvizsgálat modellek betanításához szükségesek. Emellett tartalmaz egy egyszerű, grafikus felhasználói felülettel (GUI) ellátott futtatható programot is, amelyben hasznos funkciók érhetők el, például a betanítás kezelőfelülete.
- *PythonService*: A nevéből adódóan ez az alprojekt tartalmazza azt a szolgáltatást, amelyben a betanított modellekkel megvalósított képelemző algoritmusok, a vektoradatbázis elérése, valamint a FastAPI végpontfüggvények találhatók. Ezek valósítják meg a teljes folyamatot: a detektálást, a kivágást, az összehasonlítást, valamint az adatbázisba való felvételt és törlést.
- *ReactNative*: A teljes frontend alprojektet tartalmazza, beleértve az összes frontend kódot és a konfigurációkat Androidra és iOS-re. A build folyamat után a különböző platformokhoz tartozó build-ek mappákba rendezve érhetők el.

3.2 Adatbázisséma

Az adatbázisom komplex projekt lévén sok táblát tartalmaz. A sémát úgy alakítottam ki, hogy minél bővíthetőbb legyen, ugyanakkor ne legyen annyira bonyolult, hogy ne lehessen követni. A lényeges részeknél a bővíthetőség és a program funkcióinak teljes testreszabhatósága volt a cél, míg a kevésbé lényeges részeknél igyekeztem kerülni a túl sok tábla hozzáadását. A sémáról a PgAdmin4 segítségével generáltam egy teljes ER diagramot. Ezt részekre bontva mutatom be, a teljes diagram pedig az B függelékben található, illetve jobb minőségben elérhető a GitHub Gist oldalon.

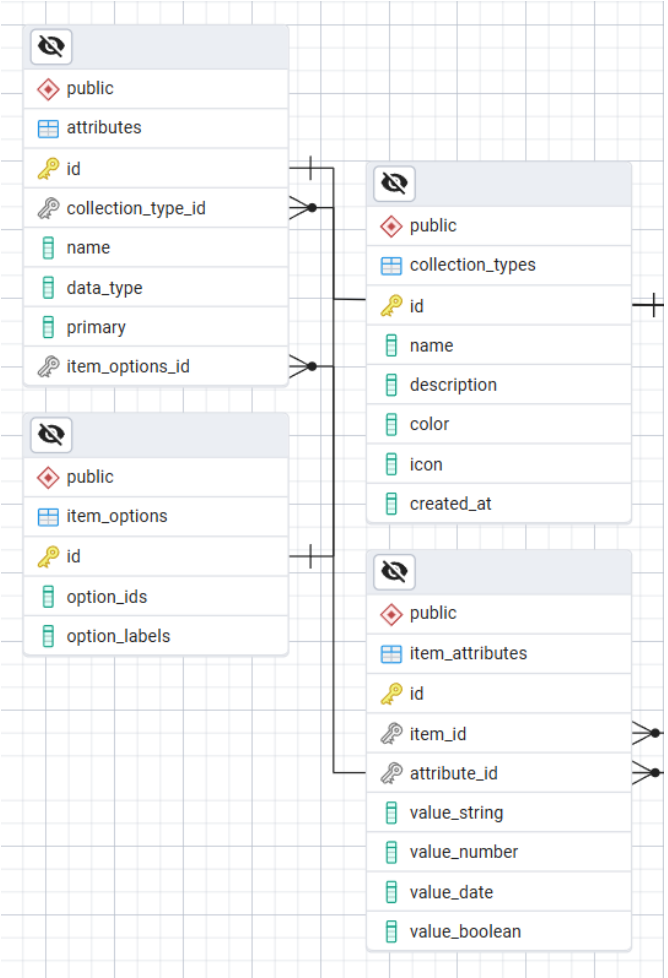
3.2.1 Users, UserContacts



17. ábra ER diagram részlet: users, user_contacts táblák

A regisztrált felhasználókat a *users* tábla tartalmazza. Az alkalmazásban a regisztráció harmadik fél (third-party) szolgáltatókon keresztül történik: a felhasználók Google- vagy GitHub-fiókkal tudnak bejelentkezni. Ennek nyilvántartására szolgál a *provider* és a *provider_user_id* oszlop. A két mező együtt egyedi, vagyis ha ugyanaz a személy több platformon (például Google és GitHub) keresztül is bejelentkezik, az új bejelentkezés új felhasználóként kerül rögzítésre az adatbázisban. A felhasználóhoz tartozik felhasználónév, ország (amely a közösségi funkcióknál kap szerepet), valamint egy *avatar* oszlop, ahová - ha létezik - a közösségi szolgáltatótól származó felhasználói avatar URL kerül. Emellett a *user_contacts* tábla tartalmazza a felhasználó elérhetőségeit, amennyiben azokat megadja. Egy felhasználónak több elérhetősége is lehet, azonban a *user_id* és a *platform* mezők együtt egyediek, így egy adott platformhoz csak egy elérhetőséget rendelhet hozzá.

3.2.2 CollectionTypes, Attributes, ItemOptions, ItemAttributes

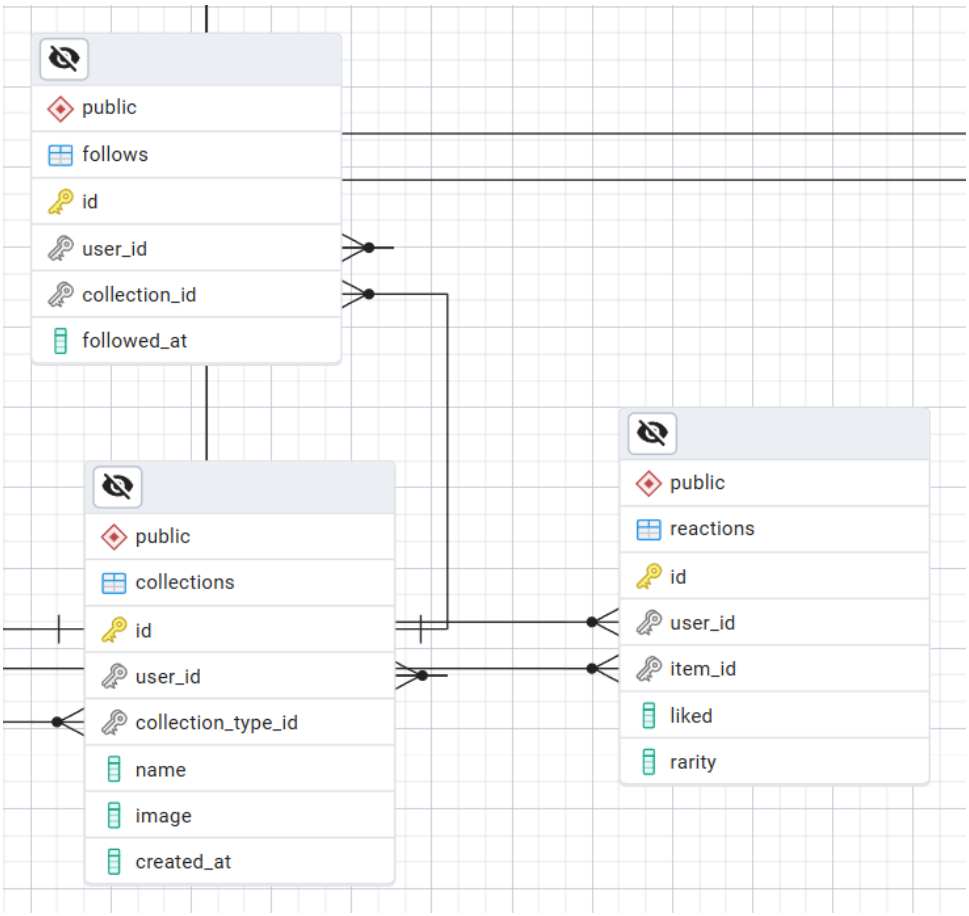


18. ábra ER diagram részlet: collection_types, attributes, item_options, item_attributes táblák

Ahogy a bevezetésben is említettem, a cél az volt, hogy az alkalmazáshoz tetszőleges tulajdonságokkal rendelkező kollekciókat lehessen hozzáadni, anélkül, hogy ehhez a backend- vagy frontend-logikán módosítani kellene. Más szóval, egy „univerzális megoldást” szerettem volna létrehozni. Ennek a megoldásnak az alapja az adatbázis sémaszintjén található. Részben az EAV modellezés (Entity-Attribute-Value) elvét alkalmaztam, elsősorban azt a tulajdonságát, hogy tetszőleges attribútumok rendelhetők egy entitáshoz. Az EAV modell lényege, hogy soronként tárolja, mely entitás mely adattípusához tartozik egy adott érték. Az entitásokat, valamint az entitásokhoz tartozó adattípusokat és azok metaadatait (például adattípus nevét) külön táblákban definiáljuk. Az ER diagrammon látható, hogy a kollekciótípusok, különböző attribútumokat tartalmazhatnak, a *collection_types* táblában szerepelnek. Itt definiálhatók az új kollekciótípusok, amelyekhez nevet, leírást, valamint a megjelenítéshez kapcsolódó metaadatokat is meg lehet adni. Ezt követően az *attributes* táblában definiálhatók az egyes

kollekciótípushoz tartozó attribútumok. Itt meg kell adni az attribútum nevét, adattípusát, valamint van egy *primary* oszlop is, amely jelzi, hogy az adott attribútum az adott kollekciótípus fő attribútuma-e. A frontend például ezt a mezőt használja arra, hogy szűrésnél a legfontosabb attribútumot előre helyezze, mivel ez várhatóan a leggyakrabban használt lesz. Ezt akár külön táblában is lehetne kezelni, de - ahogy korábban is említettem - a célom az volt, hogy ne növeljem tovább a táblák számát és a séma komplexitását. Az *item_options* tábla, arra való, hogy az attribútum típus lehet select típusú, vagyis támogatja a választós adattípust, ide kerülnek az opciók azonosítói és címkéi. Itt sem hoztam létre külön táblát az egyes opcióhalmazokhoz, mert az bonyolultabbá tette volna az adatfeltöltést, és növelte volna a rendszer komplexitását. Ehelyett az a konvenció, hogy az opciók egyetlen sorban szerepelnek, és ; jellel vannak elválasztva. Ez szerintem egyszerű és hatékony megoldás, hiszen egy *split* függvénnyel könnyen feldolgozhatók az értékek, anélkül, hogy a logika túlbonyolódna. Végül az *item_attributes* tábla az egész megoldás legfontosabb eleme. Ez rendeli hozzá a konkrét kollekcióelemekhez a tulajdonságaikat. Az *item_id* az adott elemhez, az *attribute_id* pedig az attribútumhoz kapcsolja az értéket. A táblában az egyes adattípusokhoz külön oszlop tartozik az értékek tárolására. A backend és frontend implementációban a *data_type* alapján egyértelműen meghatározható, hol található a keresett adat, és mi a típusa. Egy tárgy attribútumai tehát úgy deríthetők ki, hogy megvizsgáljuk, mely kollekcióhoz tartozik, annak milyen kollekciótípusa van, és az adott kollekciótípushoz milyen attribútumok vannak definiálva.

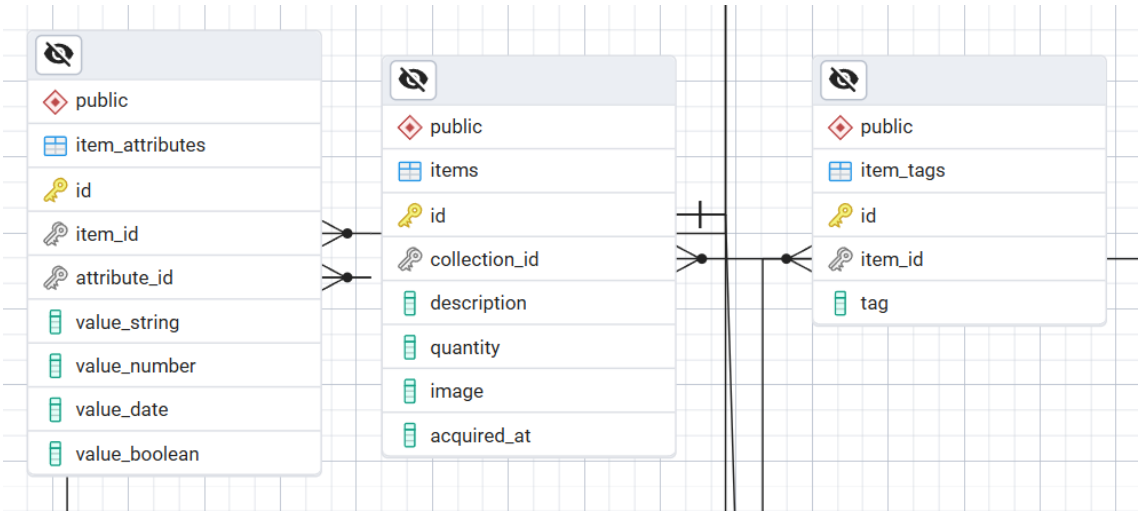
3.2.3 Collections, Follows, Reactions



19. ábra ER diagram részlet: collections, follows, reactions táblák

A *collections* tábla tartalmazza a felhasználók kollekcióit. A felhasználót a *user_id*, a kollekció típusát pedig a *collection_type_id* azonosítja. Egy felhasználónak tetszőleges számú kollekciója lehet, akár ugyanabból a típusból is. Ezáltal a felhasználó már kollekciószinten is kategorizálhatja a gyűjteményét, több kisebb részgyűjteményre bontva azt. A kollekciókhoz tartozik egy név, valamint egy opcionális borítókép, amelynek URL-je kerül eltárolásra. A másik két tábla a közösségi funkciók megvalósításához készült. Egyrészt egy kollekciót más felhasználók is követhetnek — ezek a követések a *follows* táblában tárolódnak. Másrészt egy felhasználó reagálhat egy másik felhasználó új szerzeményére: például kedveléssel (like), vagy a tárgy ritkaságának értékelésével. A *reactions* tábla tartalmazza a reagáló felhasználó azonosítóját, a tárgy azonosítóját, valamint az értékelési adatokat.

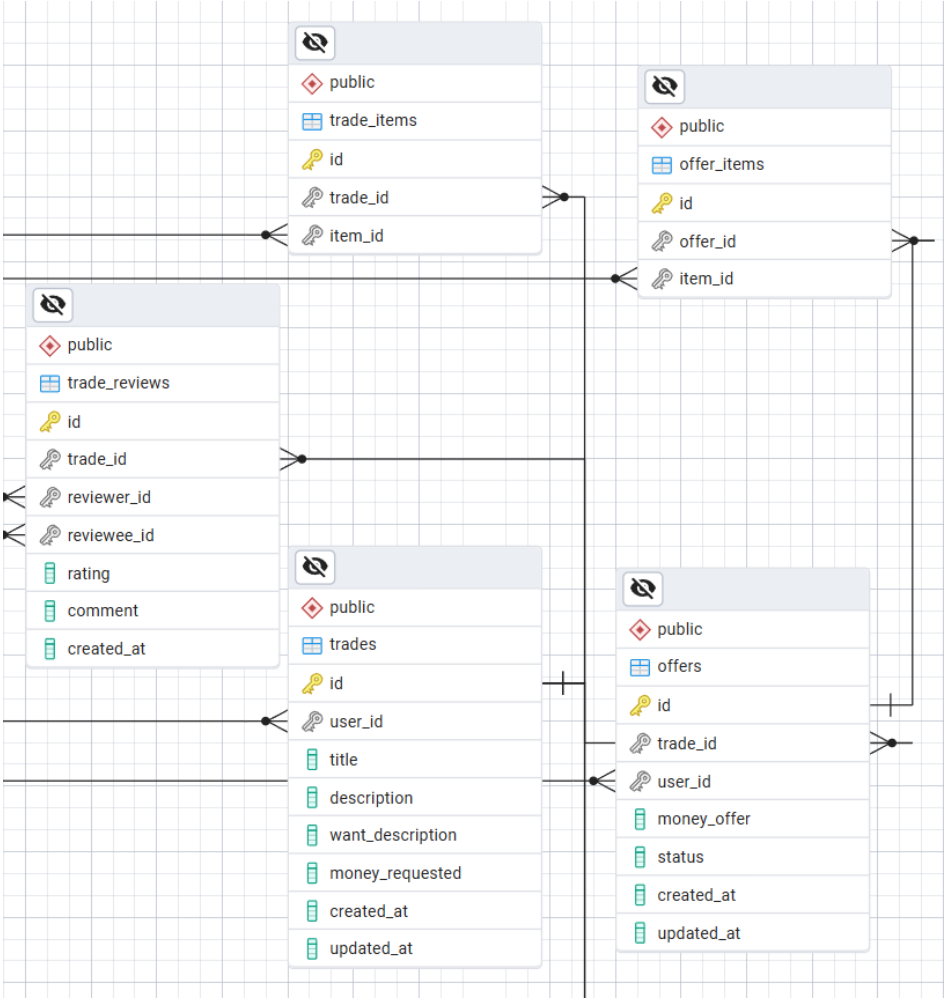
3.2.4 Items, ItemTags



20. ábra ER diagram részlet: items, item_tags, item_attributes táblák

A kollekciókhoz tartozó tárgyakat az *items* tábla tartalmazza. A *collection_id* szerint kapcsolódnak egy kollekcióhoz. Tartalmazhatnak leírást, valamint egy tárgyról a *quantity* attribútummal jelezhetjük, hogy fizikálisan mennyi van belőle, így ha több van egy tárgyból, például a csere-bere közösségi funkciónál cserére feltehetjük. Az *image* attribútum kötelező mező, és a kivágott kép URL-jét tartalmazza - ez az URL akkor kerül eltárolásra, ha a kép a detektálási folyamat után új tárgyként került elfogadásra és felvételre a kollekcióba. A tárgyakhoz lehet írni #hastag-eket, csak úgy mint más közösségi alkalmazásokban, ez az *item_tags* táblába kerül be, a tag indexelve van, ez felgyorsítja az alkalmazásban a hastag alapján filterezést. Az *item_attributes* tábla már bemutatásra került a 3.2.2-es fejezetben itt csak szemléltetve van, hogy az abba lévő értékek az *item_id* alapján kapcsolódnak hozzá a tárgyakhoz.

3.2.5 Trades, TradeItems, Offers, OfferItems, TradeReviews



21. ábra ER diagram részlet: trades, trade_items, offers, offer_items, trade_reviews táblák

Szintén a közösségi funkciók, azon belül is a csere-bere lehetőség megvalósításához készültek ezek a táblák. A felhasználók úgynevezett „trade”-eket hozhatnak létre, amelyek a *trades* táblában tárolódnak. Itt megadhatják a csere címét, leírását, valamint azt, hogy mit szeretnének kapni cserébe - akár más tárgyakat, akár pénzt. A csereajánlatokhoz („trade”-ekhez) hozzárendelhetők a cserére felajánlott tárgyak, amelyek a *trade_items* táblában kerülnek eltárolásra, és kapcsolatot teremtenek a konkrét tárgyak és a „trade” között. Egy „trade”-re más felhasználók ajánlatokat tehetnek, ezek az ajánlatok az *offers* táblában tárolódnak, és mindig egy adott „trade”-hez kapcsolódnak. Itt a *user_id* és *trade_id* együtt egyedi kulcsot alkotnak, vagyis egy felhasználó egy adott „trade”-re csak egy ajánlatot küldhet. Az ajánlat rekordja tartalmazhat pénzbeli ajánlatot is, valamint egy *status* mezőt, amely az ajánlat állapotát jelöli (ennek részleteit korábban ismertettem). Az *offer_items* tábla hasonló módon

működik a *trade_items* táblához, azzal a különbséggel, hogy az ajánlatokhoz tartozó tárgyakat tartja nyilván. Végül a *trade_reviews* tábla tartalmazza a felhasználók egymásról írt értékeléseit egy-egy „trade” lezárását követően. Egy rekord egy „trade”-hez tartozik, és két felhasználóhoz kapcsolódik: az értékelőhöz és az értékelthez. Az értékeléshez szöveg és egy bináris érték is megadható, amely azt jelzi, hogy a csere pozitív vagy negatív élmény volt-e.

3.3 Vektor adatbázisséma

A 2.7.1-es fejezetben bemutattam az embeddingek lényegét. Fontos tudni, hogy az embeddingek számítása saját gépen erőforrás-igényes lehet, online modellek használata pedig költséges, ezért ezt minimalizálni kell. Emiatt a már felvett kollekcióelemekhez nemcsak a képüket tároljuk; bár az embedding újraszámításával össze lehetne hasonlítani az új elem képével, ez lassú és nem kifizetődő. Sokkal egyszerűbb, ha az embeddinget elmentjük egy vektorok tárolására kitalált adatbázisba. Így az újonnan felvenni kívánt elem képét csak egyszer kell vektorizálni, majd összehasonlítani a vektoradatbázisban tárolt többi vektorral. A Python Service-ben a Chroma vektoradatbázist használom, amely nagyon egyszerűen kezelhető, és külön szerver futtatása nélkül elérhető a service-n keresztül. A vektorok felvételekor az adatbázisban tárolt rekordokhoz az alábbi metaadatokat rendeljük: az elem ID-ját, valamint a kollekciót, amelyhez tartozik. Így például új elem hozzáadásakor csak az adott kollekcióhoz tartozó vektorokkal hasonlítjuk össze az új vektort. Ez további segítő funkciókat is lehetővé tesz, például kollekció törlése esetén a kollekció ID alapján könnyen azonosíthatók a törlendő vektorok.

A vektoradatbázis sémája:

vectors(a vektorokat tartalmazó vektor adatbázis)

- *id*: megegyezik a vektorhoz tartozó tárgy relációs adatbázisbeli azonosítójával
- *value*: maga a vektor értéke
- *collectionId*: az elemhez tartozó kollekció azonosítója

3.4 Betanított modellek konvenció

Az alkalmazás úgy működik, hogy minden új kollekciótípushoz szükség van két betanított modellre: egy tárgydetektáló és egy vektorizáló modellre. Ezek a modellek a 3.1-es fejezetben bemutatott *PythonService* projekt részei, és a projekt *models* mappájába kerülnek, az adott kollekcióhoz tartozó modell nevében a kollekció ID-jával. Innen tudja a service, hogy melyik modellt kell meghívni.

A konvenció szerint a két modell neve a következő:

- *<collectionId>_detection.pt* (a tárgydetektáló modell)
- *<collectionId>_similarity_<classNumber>.pt* (a hasonlóságmodell)

A hasonlóság modellhez, a fájl végére kell fűzni, hogy mennyi osztályon tanult, mert azt meg kell határozni a használat előtt az inicializlásánál.

3.5 „Univerzális” megoldás összefoglalás

A fentiekben mindent ismertettem ahhoz, hogy megérthető legyen, miként sikerült megvalósítani az „univerzális megoldást”, vagyis azt, hogy tetszőleges attribútumokkal rendelkező kollekciótípusokat vehessünk fel. Természetesen ez csak az alapot adja: ez teremti meg a funkció technikai lehetőségét, de ahhoz, hogy ez ténylegesen működjön az alkalmazásban, a backendnek és a frontendnek is támogatnia kell az attribútumok típusainak megfelelő feldolgozást és megjelenítést.

Az alábbi táblázat foglalja össze, milyen lépések szükségesek egy új kollekció felvételéhez.

új kollekciótípus felvétele a *collection_types* táblába	új attribútumok felvétele az új kollekciótípushoz az *attributes* táblába	új tárgydetektáló modell betanítása, és fevétele a *PythonService models* mappájába a konvenció szerint	új vektorizáló modell betanítása, és felvétele a *PythonService models* mappájába a konvenció szerint

4 Projekt részletezése

Az előző fejezetekben megismertük a projekt célját, funkcióit, technológiáit, valamint alapvető felépítését adatbázisséma szinten. Mivel mindezzel már tisztában vagyunk, most ezekre alapozva bemutatom a projekt érdekesebb és bonyolultabb részleteit, a főbb szerveroldali és frontend technológiák szerint tagolva.

4.1 ASP.NET Core backend

4.1.1 Third party bejelentkezés, és JWT szolgáltatás

A program egyik lényeges része, amellyel sokat foglalkoztam, az autentikáció megfelelő működésének biztosítása volt. Korábban a hobbiprojektjeimben és egyetemi házi feladataimban mindig saját regisztrációs és bejelentkezési megoldásokat készítettem, azonban ezek valós üzleti környezetben nem lettek volna biztonságosak. Emellett mindig is érdekelt, hogyan működik a külső (third party) autentikáció. Úgy gondoltam, hogy egy teljesen saját, hibamentes autentikációs rendszer megvalósítása nehezebb feladat lenne, mint egy külső szolgáltató integrálása, és mivel ez a téma különösen érdekelt, ebben a projektben ezt a megközelítést választottam. Hozzá kell tennem, hogy ez sem bizonyult egyszerűnek, de rengeteget tanultam belőle.

Az autentikációs folyamat két részre bontható. A ritkábban használt rész maga a külső fél általi autentikáció, majd a beléptetés vagy regisztráció. Ezt követően a bejelentkezett felhasználó már a másik folyamatot, a belső autentikációt használja mindaddig, amíg explicit módon ki nem lép az alkalmazásból. Ezt a belső autentikációt a JWT (JSON Web Token) alapú szolgáltatás valósítja meg. Amennyiben a felhasználó be van jelentkezve, az alkalmazás minden indításakor a kijelentkezésig, JWT tokennel történik a hitelesítése. Az alábbiakban bemutatom a két folyamat működését részletesen.

```
public record TokenRequest(string IdToken);
[HttpPost("google/validate")]
public async Task<IActionResult> GoogleValidate([FromBody] TokenRequest req)

public record RegisterRequest(string TempId, string Username, string Country);
[HttpPost("google/register")]
public async Task<IActionResult> GoogleRegister([FromBody] RegisterRequest req)

public record GithubCodeRequest(string Code, string CodeVerifier);
[HttpPost("github/validate")]
public async Task<IActionResult> GithubValidate([FromBody] GithubCodeRequest req)

public record RegisterRequestGithub(string TempId, string Username, string
Country);
[HttpPost("github/register")]
public async Task<IActionResult> GithubRegister([FromBody] RegisterRequestGithub
req)
```

22. ábra Külső autentikációhoz használt API-függvények fejlécei

Külső autentikálás két esetben lehetséges: ha a felhasználó először lép be az applikációba, vagy ha a beállítások menüben kiléptette magát. A kiléptetés úgy működik, hogy az applikáció törli a telefon lokális tárhelyéből az automatikus, belső autentikációhoz szükséges JWT-tokent. Ilyenkor belső autentikáció nélkül nem lehet lekérni az aktuális felhasználót, ezért a bejelentkezéshez valamely külső autentikációs módot kell választani. Amikor a felhasználó kiválasztja valamelyik third party autentikációt, és végigmegy a folyamaton, kap egy rövid élettartamú *IdToken*-t (vagy például a GitHub esetében egy *Code* és *CodeVerifier* párost). Ez a külső szolgáltatótól függ, de a lényeg, hogy ezek rövid élettartamú azonosítók, amelyek segítségével a backend a megfelelő third party könyvtár használatával autentikálja a felhasználót. Első lépésként mindig a 22. ábra által szemléltetett */validate* végpontok valamelyike hívódik meg. Ez megpróbálja a külső szolgáltatón keresztül beléptetni (autentikálni) a felhasználót, majd a szolgáltató típusa és a szolgáltatótól kapott egyedi felhasználó-azonosító alapján megkeresi őt az adatbázisban. Ha a felhasználó létezik, a backend generál számára egy JWT-tokent, amelyet az applikáció ezt követően a belső autentikációhoz használhat. Ha nem létezik, a backend visszaadja, hogy a felhasználó nem található, valamint küld egy ideiglenes azonosítót. Ezt követően az applikáció meghívja a 22. ábra szerinti */register* végpontok valamelyikét, amely paraméterként megkapja a felhasználónevet, az országot és az ideiglenes azonosítót. Ezek hiányában a backend elutasítja a regisztrációs kérelmet. Sikeres regisztráció esetén a felhasználó bekerül az adatbázisba, és kap egy JWT-tokent, amellyel a későbbiekben már a belső autentikációs folyamaton keresztül azonosítja magát.

```
if (!_tempTokenService.TryGetToken(req.TempId, out var accessToken))
{
    return BadRequest("Invalid registration session");
}

var payload = await GoogleJsonWebSignature.ValidateAsync(accessToken);

var existingUser = await _usersRepository.GetUserByLogin("google",
payload.Subject);
if (existingUser != null)
{
    return BadRequest("User already exists");
}

var user = await _usersRepository.InsertUserAsync(new UserRequired("google",
payload.Subject, req.Username, payload.Email, req.Country, payload.Picture));

var jwt = _jwtService.GenerateToken(user);

_tempTokenService.RemoveToken(req.TempId);

return Ok(new { token = jwt });
```

23. ábra Google autentikáció regisztrációs folyamat

A 23. ábra az előbb ismertetett regisztrációs logika függvényrészletét mutatja. Látható benne három nagyon fontos elem: a visszautasítás, ha az ideiglenes azonosító érvénytelen; a *GoogleJsonWebSignature* autentikációs függvény a third party könyvtárból; valamint a JWT token generálása. Érdekesség, hogy a WebSignature autentikálás után a *payload*-ban számos felhasználói információhoz hozzáférünk, mint például a szolgáltatón belüli egyedi azonosító, az e-mail-cím vagy a profilkép URL-je, amelyet fel is használok, és elmentek az adatbázisba, hogy a felhasználókhoz profilképet tudjak megjeleníteni.

A külső bejelentkezés után a felhasználó telefonján már megtalálható a JWT-token, így a továbbiakban azon keresztül azonosítja magát az alkalmazás indításakor. Ez a felhasználó számára láthatatlan, automatikus beléptetést jelent.

```
public string GenerateToken(User user)
{
    var claims = new[]
    {
        new Claim(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
        new Claim(JwtRegisteredClaimNames.Email, user.Email),
        new Claim("provider", user.Provider)
    };

    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_secret));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        issuer: _issuer,
        audience: _issuer,
        claims: claims,
        expires: DateTime.UtcNow.AddYears(1),
        signingCredentials: creds
        );


    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

24. ábra JWT-token generáló függvény

A külső beléptetés során használt JWT-token generáló függvényt a 24. ábra szemlélteti. A JWT-tokenekbe állításokat ágyazhatunk, jelen esetben a felhasználó adatait. Ezután létrehozunk egy aláírást, amelyet saját privát kulcsunkkal kódolunk. Ezt követően kiállítjuk az általunk aláírt JWT-tokent. Az így kiállított, titkosított tokent visszafejtve megkaphatjuk a hordozott hiteles állításokat, amelyeket felhasználhatunk a bejelentkezés során. A tokenekhez lejárati dátumot is rendelhetünk, amely után érvényüket vesztik; az én implementációmban ez egy év a kiállítás után. Üzleti programokban a biztonság miatt ez sokkal rövidebb, és a munkamenet közben gyakran frissítik.

```
services.AddAuthentication("Bearer")
    .AddJwtBearer("Bearer", options =>
     {
         options.TokenValidationParameters = new TokenValidationParameters
         {
             ValidateIssuer = true,
             ValidateAudience = true,
             ValidateLifetime = true,
             ValidateIssuerSigningKey = true,
             ValidIssuer = "coffer.backend",
             ValidAudience = "coffer.backend",
             IssuerSigningKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(Env.GetString("JWT_SECRET") ?? throw
new InvalidOperationException("JWT_SECRET envionmental variable is not set")))
         };
     });
services.AddAuthorization();
```

25. ábra ASP.NET Core beépített autentikáció konfigurációja

Az ASP.NET Core beépítetten támogatja a végpontok elérésének felhasználó általi azonosítását és az engedélyezést. Ehhez a végpont függvény fölé az *[Authorize]* attribútumot kell írni, valamint konfigurálni kell a szervert az azonosításra. Ehhez az *AddAuthentication()* függvényt használjuk, ahogy azt a 25. ábra szemlélteti. Az implementációm szerint a HTTP-kérés fejlécében egy Bearer JWT-tokennek kell szerepelnie, amelyet validálunk több szempontból is, például a kiállító, az igénylő, az élettartam, valamint a hiteles aláírás alapján. Végül megadjuk a saját privát kulcsot, amivel a tokent dekódolni lehet – ez az a kulcs, amivel kódoltuk is. Az *AddAuthorization()* függvénnyel hozzáadjuk az engedélyezés szolgáltatását, hogy a végpontok elérhetőségét az autentikáció eredménye alapján lehessen szabályozni.

```
[HttpGet("currentUser")]
[Authorize]
public async Task<ActionResult<User>> GetCurrentUser()
{
    var userIdClaim = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;

    if (userIdClaim == null)
        return Unauthorized();

    var userId = Guid.Parse(userIdClaim);

    var user = await _usersRepository.GetItemByIdAsync(userId);

    if (user == null)
        return NotFound();

    return Ok(user);
}
```

26. ábra Az aktuális felhasználó lekérése authorizációval

A belső azonosítás során mindig a 26. ábra által szemléltetett végpont függvény hívódik meg. Ha nincs érvényes *Bearer* JWT-token a HTTP-fejlécben, a kérés automatikusan elutasításra kerül. Ha van token, a függvény ellenőrzi, hogy tartalmaz-e állítást a felhasználó egyedi adatbázisbeli azonosítójáról; ha nincs, szintén *Unauthorized* választ ad. Ha az azonosító megtalálható, a függvény lekéri az adatbázisból a hozzátartozó felhasználót, és visszaadja azt; ellenkező esetben *Not Found* választ kapunk. Az applikáció oldaláról ez úgy néz ki, hogy a *Bearer* token a fejlécben az a JWT-token, amelyet a telefon lokális tárhelyén tárol.

4.1.2 Generikus CRUD

A fejlesztés során fontos szempont volt számomra, hogy minimalizáljam a felesleges ismétlődő munkát: ahol ugyanaz a logika, ott ne írjam meg többször, mert ez időigényes, és növeli a hibalehetőséget. Ezért használtam ki a generikus osztályok és függvények előnyeit, így sikerült egy nagy, logikailag összefüggő részt általánosítani, és az új CRUD-műveletek létrehozását adott adatbázisosztályokra framework-szerűen megvalósítani.

```
public interface IGenericEntity<Tkey> where Tkey : notnull
{
    Tkey Id { get; set; }
}
```

27. ábra Entitás osztályok megvalósítandó interfésze

Először létrehoztam egy interfészt, amint azt a 27. ábra mutatja. Ezt az interfészt az entitásosztályokhoz rendeltem, biztosítva, hogy minden objektumnak legyen egyedi azonosítója, amely valószínűleg az adatbázisból kerül vissza. Így függetlenül attól, hogy a CRUD generikusan van megvalósítva a repository-kban és a controllerekben, az *Id*-hez, amely bármilyen típusú lehet, mégis hozzá tudok férni.

```
public interface IReadOnlyGenericRepository<Tkey, TEntity, TProvided>
    where Tkey : notnull
    where TEntity : TProvided
    where TProvided : class, IGenericEntity<Tkey>
{
    Task<IEnumerable<TProvided>> GetItemsAsync(string? filter = null, string?
orderBy = null, int? page = null, int? pageSize = null);
    Task<TProvided?> GetItemByIdAsync(Tkey id);
}
```

28. ábra Generikus csak olvasható repository interfész

Mivel a repository mintát követem, a generikus repository is interfészként készült, amint azt a 28. ábra mutatja. Ez az interfész csak olvasható (read-only) függvényeket tartalmaz. Ezt a CRUD-tól való szétválasztást azért választottam, hogy azoknak az adatbázis-entitásoknak a kontrollerei, amelyeket a felhasználó nem módosíthat, csak olvashat, ne szolgáljanak ki feleslegesen változtató végpontokat. Az interfész előírja, hogy legyen lehetőség minden adatbázis-entitást visszaadni, valamint visszaadható legyen entitás Id szerint. A generikusság úgy valósul meg, hogy meg kell adni az *Id* kulcs típusát, valamint a teljes adatbázisbeli leképezést *TEntity*-ként. A *TEntity* a *TProvided*-ból származik, amely megvalósítja az előbb ismertetett *IGenericEntity* interfészt, így biztosítva az *Id* meglétét. A *TEntity* osztály az adatbázisbeli leképezésre szolgál, így a repository tudja, melyik *DbSet*-tel kell dolgozni. A *TProvided* osztály pedig DTO-ként funkcionál a GET végpontok számára, mert előfordulhat, hogy az entitásnak vannak olyan attribútumai, amelyek nem szükségesek az applikáció számára, ezért ezeket nem adjuk vissza. Például a felhasználó *ProviderUserId* tulajdonsága nem szükséges a frontendnek, csak a backend használja, így ez nem kerül bele a *TProvided*-ba.

```
public interface IGenericRepository<Tkey, TEntity, TProvided, TRequired>
: IReadOnlyGenericRepository<Tkey, TEntity, TProvided>
    where Tkey : notnull
    where TEntity : class, TProvided
    where TProvided : class, IGenericEntity<Tkey>
    where TRequired : class
{
    Task<TProvided> InsertItemAsync(TRequired item);
    Task<TProvided> InsertItemAsync(TProvided item);
    Task<TProvided?> UpdateItemAsync(Tkey id, TRequired item);
    Task<TProvided?> UpdateItemAsync(Tkey id, TProvided item);
    Task<bool> DeleteItemAsync(Tkey id);
}
```

29. ábra Generikus repository interfész

Ez a repository interfész azon kontrollerek számára készült, ahol módosító végpontokat is implementálni kell. Az interfész hasonlóan működik, mint a read-only változat, hiszen azt is implementálja, ugyanakkor támogatja az *insert*, *update* és *delete* funkciókat. A korábbi három generikus paraméterhez hozzáadódott a *TRequired*, amely DTO-ként szolgál a változtató végpontok számára. Ennek célja, hogy a frontendnek ne kelljen minden adatot feltöltenie, csak azok kerüljenek át, amelyek feltétlenül szükségesek. Így például nem kell ideiglenes *Id*-t létrehozni a frontenden, vagy a bonyolult *CreatedAt* dátumot csatolni, mert ezekről a backend gondoskodik. A változtató függvényekből két fajta létezik: a *TRequired* paraméterű, amely gyakran használt, és a

TProvided alapú, amely szintén támogatott ha esetleg ilyen típus jönne vissza a frontendről, így biztosítva a rugalmasságot a CRUD műveletek során.

A fent bemutatott generikus interfészeknek elkészültek a generikus implementáció, ugyanis így minden adott ahhoz, hogy akkor az implementáció is generikus legyen. Csupán egy valami szügséges minden különböző entitás repository implementációjakor. Egy Map függvény.

```
protected abstract TEntity MapToEntity(TRequired required, TEntity? entity =
null);
protected abstract TEntity MapToEntity(TProvided provided, TEntity? entity =
null);
```

30. ábra Leképező függvények

Az *IGenericRepository* interfész a *GenericRepository* osztályban van megvalósítva. A konkrét implementációban azonban van néhány olyan rész, amit a generikusság önmagában nem tud kezelni. Ez abból adódik, hogy az adatbázisba adatot *TEntity*-ként kell feltölteni, míg a változó vagy új adat a *TRequired* vagy *TProvided* típusban érkezik. Ezeket az adatokat a konkrét repository implementációjában kell *TEntity*-re képezni, hogy fel lehessen tölteni az új vagy módosított adatot. Ez a plusz logika minden esetben az adott entitás repository-jában valósul meg, és itt kell felülírni a szükséges függvényeket, például a *CreatedAt* dátum kezeléséhez. A felülirandó függvényeket a 30. ábra mutatja.

Az *IReadOnlyGenericRepository* interfész a *ReadOnlyGenericRepository* osztályban van megvalósítva. Itt nincs szükség plusz információra, és nem kell külön repository interfészt vagy osztályt létrehozni az adott adatbázis-entitás számára. Egyszerűen a *ReadOnlyGenericRepository*-t paraméterezve létre tudunk hozni read-only repository-kat.

Ha egy adott repository-nak a CRUD műveleteken túl további, speciális adatbázis-függvényeket kell megvalósítania, akkor az interfészét leszármaztatjuk valamelyik generikus repository interfészből. A saját interfészben definiáljuk a szükséges extra függvényeket, majd a konkrét repository implementációban valósítjuk meg ezeket a leképező függvényekkel együtt.

```
[ApiController]
[Route("api/[controller]")]
public class GenericController<TKey, TEntity, TProvided, TRequired>
:ReadOnlyGenericController<TKey, TEntity, TProvided>
    where TKey : notnull
    where TEntity : class, TProvided
    where TProvided : class, IGenericEntity<TKey>
    where TRequired : class
```

31. ábra Generikus kontroller osztály

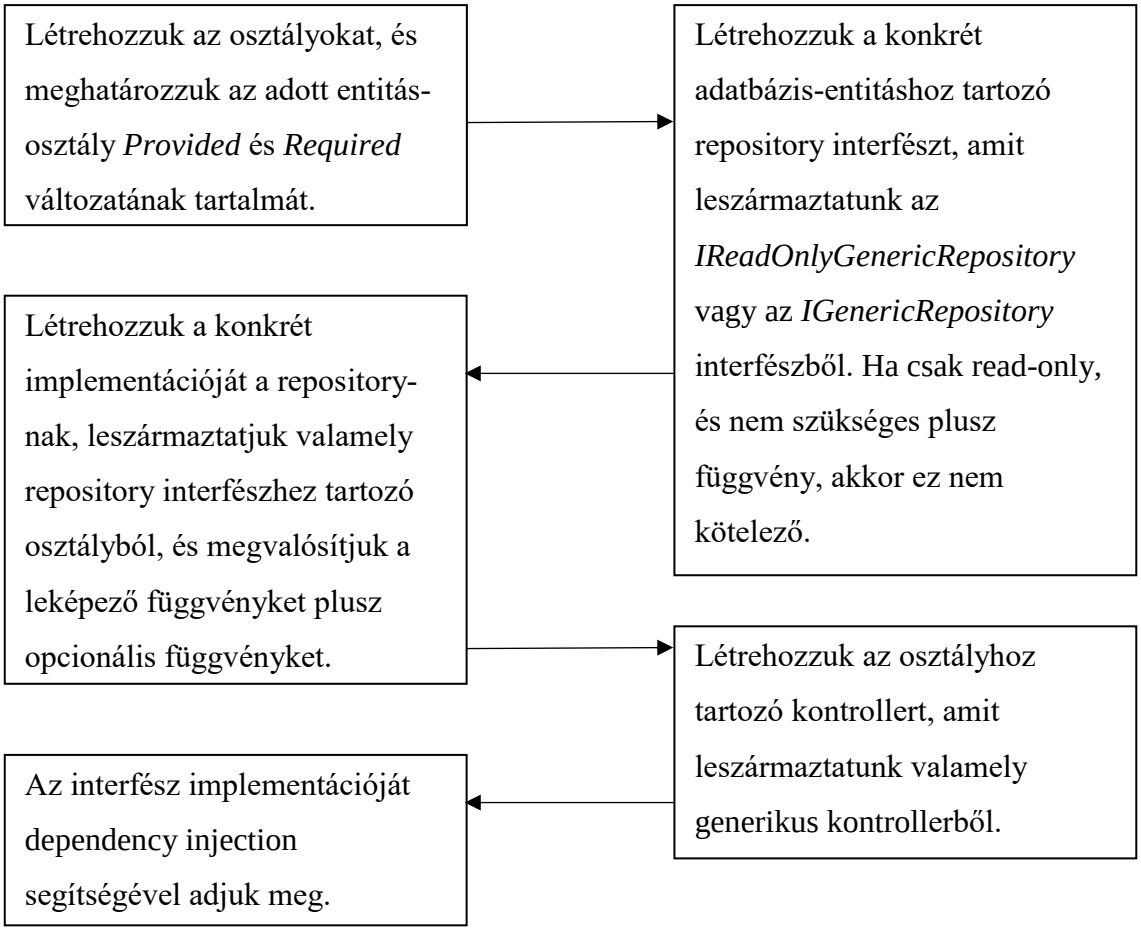
A repository után a kontroller szinten is megvalósítottam a generikusságot, mind read-only, mind módosító műveletek esetén, ahogy a 31. ábra szemlélteti a *GenericController* osztályon keresztül. Ez a kontroller a generikus függvényeket hívja meg az adott végponthoz tartozó logika szerint; hiba esetén a megfelelő hibát dobja, egyébként elvégzi a műveletet és sikeres jelzést ad.

```
[Route("api/[controller]")]
public class UserContactController : GenericController<Guid, UserContactProvided,
UserContactProvided, UserContactRequired>
{
    public UserContactController(IGenericRepository<Guid, UserContactProvided,
UserContactProvided, UserContactRequired> repository) : base(repository)
    {
    }
}
```

32. ábra Egyszerű kontroller implementáció

Az egész generikusság azt eredményezi, hogy egy egyszerű CRUD-műveleteket tartalmazó kontrollert minimális kóddal meg lehet valósítani, ahogy azt a 32. ábra mutatja.

Az alábbiakban bemutatom, hogyan lehet ezt a módszert framework-szerűen használni:



4.2 React Native

4.2.1 Expo app.json

A 2.4.2-es fejezetben már ismertettem az Expo eszköztárat. Az egyik leghasznosabb tulajdonsága - amellyel sokat foglalkoztam a megfelelő konfiguráció miatt - az *app.json* fájl. Ez egy központi konfigurációs állomány, amelyben egyszerre tudunk beállításokat megadni Androidra, iOS-re és webre. Ennek nagy előnye, hogy nem kell mindhárom platform konfigurációját külön-külön módosítani: ha az app.json-ben megadjuk a beállításokat, akkor a buildelés során automatikusan létrejönnek a megfelelő platformspecifikus konfigurációs fájlok. Emellett ebben a JSON-ben megadhatunk kizárólag egy adott platformra vonatkozó beállításokat is, így a teljes konfiguráció központilag, egyetlen helyen kezelhető.

```
"android": {
  "package": "com.gelomelo.coffer",
  "intentFilters": [
    {
     "action": "VIEW",
        "data": [
         {
           "scheme": "com.gelomelo.coffer"
         }
      ],
     "category": ["BROWSABLE", "DEFAULT"]
    }
  ],
},
```

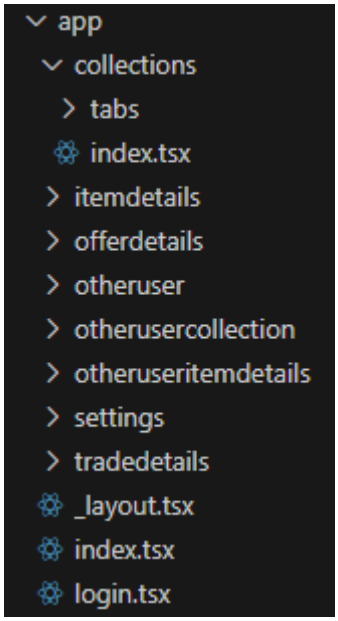
```
<intent-filter>
  <action android:name="android.intent.action.VIEW"/>
  <category android:name="android.intent.category.DEFAULT"/>
  <category android:name="android.intent.category.BROWSABLE"/>
  <data android:scheme="com.gelomelo.coffer"/>
  <data android:scheme="exp+coffer"/>
</intent-filter>
```

33. ábra app.json android konfiguráció (részlet), alatta ugyanez az AndroidManifest.xml fájlban

A 33. ábra mutatja a konfiguráció példáját. A felső kódrészlet a React Native projekt *app.json* fájljából származik, míg az alsó rész az Android build során automatikusan legenerált *AndroidManifest.xml* megfelelő szakasza. Jól látható, hogy a build folyamat az app.json-ben megadott beállítások alapján hozza létre az Android-specifikus konfigurációt: ugyanazok az intent filterek és adatsémák kerülnek átemelésre, amelyeket a központi konfigurációban meghatároztunk. A kód feladata, hogy az alkalmazás számára egyedi URL-sémákat (deep linkeket) definiáljon. Ennek segítségével a rendszer képes felismerni, ha egy olyan linket nyitunk meg - például *com.gelomelo.coffer://…* - amelyet az alkalmazásnak kell kezelnie, és automatikusan az appot indítja el a megfelelő képernyőre navigálva. Ez a mechanizmus teszi lehetővé többek között a külső autentikáció utáni visszairányítást vagy különböző mélylinkek kezelését.

4.2.2 Expo router

Az Expo egyik nagy újítása a React Native hagyományos, bonyolultabb navigációs folyamatának jelentős leegyszerűsítése. A korábbi navigációs könyvtárak használata sok kézi konfigurációt igényelt: külön stackeket kellett definiálni, manuálisan kellett beállítani az útvonalakat, és konfigurálni a mélylinkeket az egyes oldalakhoz. Ezzel szemben az Expo Router használatával mindez lényegesen egyszerűbbé válik, mivel konvencióalapú megközelítést alkalmaz. Az Expo Router a */app* mappát tekinti kiindulópontnak, és az ezen belül létrehozott fájlokhoz automatikusan útvonalat rendel: például a */app/login.tsx* fájl elérési útvonala automatikusan */login* lesz. Ennek köszönhetően nincs szükség stackek és útvonalak kézi definiálására, mert a navigáció teljesen a mappa-struktúrára épül. Ez a megoldás sokkal átláthatóbbá és gyorsabbá teszi a fejlesztést, én pedig pontosan ezért ezt a módszert használtam.



34. ábra Expo router-hez használt mappastruktúra

A mappastruktúrát az oldalakhoz én is az Expo Router konvenciói alapján alakítottam ki, ahogyan azt a 34. ábra is szemlélteti. Az azonos mélységben található fájlok az alkalmazás útvonalai között is ugyanolyan mélységet képviselnek, de természetesen lehetőség van további beágyazásra. A projektben például az *index.tsx* és a *login.tsx* a gyökérszinten találhatóak, és ezek határozzák meg a kezdőoldalt, illetve a bejelentkező oldalt. Minden mappaszinten létrehozható egy *_layout.tsx* fájl, amely az adott szint alap struktúráját definiálja - például hogy legyen-e felső vagy alsó navigációs sáv. Ha egy mélységben nincs megadva, hogy pontosan melyik oldalra navigáljon, akkor

a rendszer automatikusan az *index.tsx*-ben meghatározott oldalra navigál. A mappastruktúrából látható, hogy a *collections* mappában is található egy *index.tsx,* azonban ezen kívül további, mélyebben beágyazott oldalak is szerepelnek. Ennek megfelelően a *collections* útvonala */collections*, míg a mélyebben lévő oldalak például */collections/tabs* (az ottani index), illetve */collections/tabs/oldalnév* formában érhetők el.

4.2.3 Zustand global store

React applikációk fejlesztéséből már ismert lehet a *prop drilling* kifejezés, amikor egy magasabb szintű szülőkomponensből több lépcsőn keresztül adjuk tovább az állapotot olyan gyermekkomponenseknek, amelyeknek szükségük van rá. Ez felesleges plusz kódot eredményez, és a kód olvashatóságát is rontja, mivel bizonyos komponensek csupán azért kapnak propként állapotot, hogy továbbadhassák azt lefelé a komponensfában. Ugyanez a probléma React Native esetében is fennáll, hiszen ugyanarra a keretrendszerre épül. React Native-ben a navigáción keresztüli adatátadás további nehézségeket okoz: a paramétereket JSON formában kell átadni az útvonalhoz csatolva. Ez bonyolulttá teszi az összetett adatstruktúrák továbbítását, mivel minden esetben konverziós logikára lenne szükség az objektum → JSON és JSON → objektum átalakításokhoz. Ezt szerettem volna elkerülni, hogy a kód tisztább maradjon és időt spóroljak. Ezért választottam a globális állapottár (global store) használatát. Ez azt jelenti, hogy van egy (vagy több) központi állapottár a teljes alkalmazásban, amely bármely komponensből elérhető egy hook segítségével, így mind az állapot, mind annak módosító függvénye könnyen hozzáférhető - *prop drilling* nélkül. Az egyik lehetséges megoldás a React beépített Context API-ja lenne, azonban ez viszonylag sok plusz előkészítést igényel: kontextusok létrehozását, az ezekhez kapcsolódó hookok megírását, valamint a megfelelő komponensek köré való wrappert. Ehelyett a Zustand könyvtárat választottam, amely nagyon egyszerűen használható, könnyen lehet vele globális store-okat létrehozni, és kiváló dokumentációval rendelkezik.

```
export interface UserStore {
  user: User | null;
  setUser: (user: User | null) => void;
}

export const useUserStore = create<UserStore>((set) => ({
  user: null,
  setUser: (user) => set({ user }),
}));

  const { user, setUser } = useUserStore();
```

35. ábra Store típus definiálása és global store létrehozása

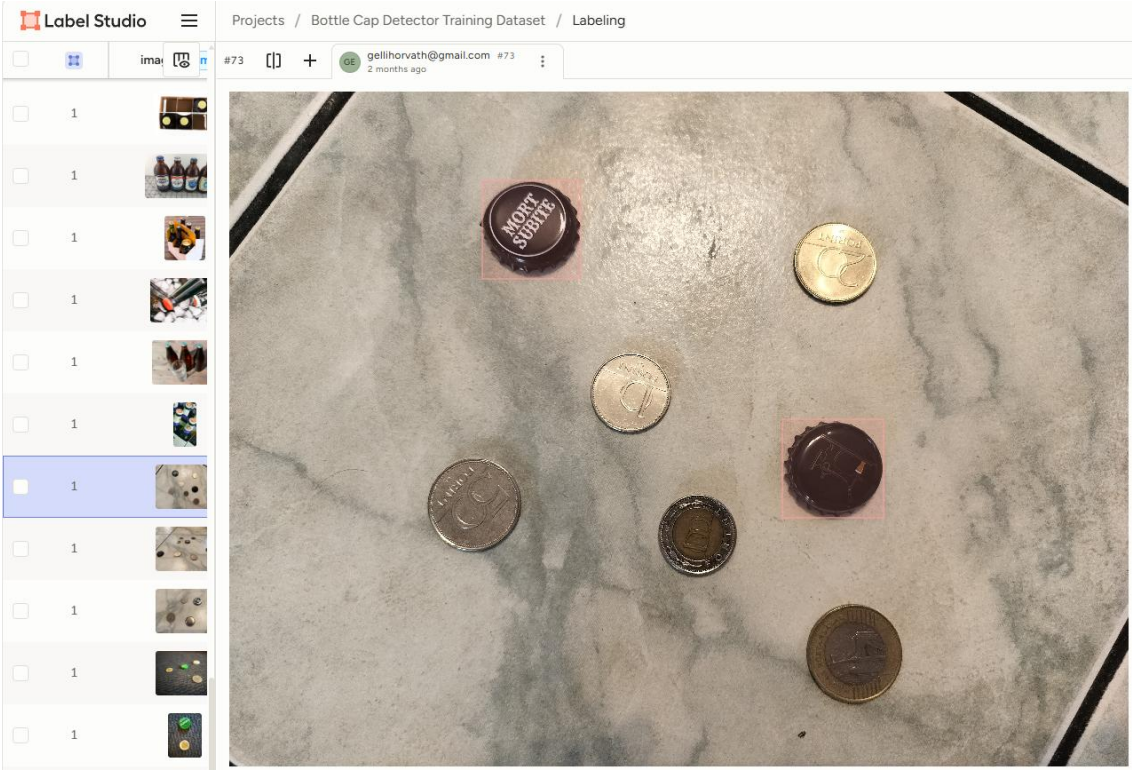
A Zustand működését a bejelentkezett felhasználó adatait tároló globális store-on keresztül mutatom be, amit a 35. ábra szemléltet. Először létre kell hozni egy interfészt, amely definiálja, hogy mi legyen elérhető a store-ból. A projektemben a *UserStore* interfész biztosítja a struktúrát a bejelentkezett felhasználó tárolásához. A bejelentkezett felhasználóra kell a global store, mivel az adatait sok helyen használom, például megjelenítésre vagy további adatok lekérésére a backendről. A *user* értéke *User* típusú lehet, vagy *null*, mivel a globális store a program futásának elején inicializálódik, amikor még nincs bejelentkezett felhasználó. Ezen kívül definiáltam egy setter függvényt is a *user* állapot frissítésére. Ezután a Zustand *create* függvényével létrehozom a globális store-t, ahol megadom az alapértelmezett értékeket: a *user* kezdetben null, a *setUser* pedig a Zustand által biztosított *set* függvény. Így egyszerűen elkészül a globális store, és innentől a *user* és a setter függvény bárhonnan elérhető a *useUserStore* hook meghívásával, ahogy a 35. ábra is mutatja.

4.3 Python service, modell betanítás

4.3.1 Tárgyfelismerő modell betanítás

A tárgyfelismerés működését már a 2.6-os fejezetben kifejtettem, és példakódot is bemutattam, amelyet a 12. ábra szemléltet. Ez a példakód pontosan ugyanaz, amelyet a kupakfelismerő modell betanításához használtam. A YOLOv8 modell szerencsére nagyon gyorsan és könnyen betanítható, mivel ténylegesen ennyi kód elegendő hozzá a

megfelelő mappastruktúra mellett, amelyet szintén ebben a fejezetben ismertettem. Én is ezt a megközelítést alkalmaztam.



36. ábra Label Studio adat címkézés

Először a Label Studio segítségével létrehoztam a betanításhoz szükséges adatokat. Ennek folyamatát a 36. ábra szemlélteti. Végigmentem az összes képen - ez 81 képet jelentett - és minden egyes képen határoló négyzettel megjelöltem a kupakokat. Ezt követően elvégeztem ugyanezt a validációs adatokon is, amelyek összesen 10 képből álltak. A validációs adatokat a modell nem tanulásra használja, hanem a tanulás sikerességének mérésére. Miután minden kép annotálása megtörtént, a Label Studio lehetőséget ad az adatok exportálására. Az exportált csomag lényegében minden képhez létrehoz egy szöveges fájlt, amely tartalmazza az adott képen szereplő határoló négyzetek koordinátáit - pontosan azt a formátumot, amelyre a YOLO betanításához szükség van. Ezt követően létrehoztam a 2.6.2-es fejezetben ismertetett mappastruktúrát, és betanítottam a modellt. A tanulás 50 epochból állt, vagyis a modell 50 alkalommal ment végig a tanítóadatokon; ez körülbelül 10 percet vett igénybe a számítógépemen. A rendszer ezt a betanított modellt használja arra, hogy a nagy képből kivágja azokat a részeket, amelyeken a céltárgy - jelen esetben a kupak - látható.

4.3.2 Embedding modell betanítás

Az embedding modellre azért volt szükség, mert egy általános célú képfelismerő és vektorizáló modell, mint például a CLIP, nem adott megfelelő eredményt: ugyanarról a tárgyról készült képek vektorai túl távol estek egymástól. Emiatt ehhez a feladathoz külön, saját modellt kellett betanítanom. Ez jóval összetettebb feladat volt, mint a tárgyfelismerő modell tanítása, és a folyamat is sokkal tovább tartott: egy-egy tesztbetanítás körülbelül 30 percet vett igénybe. Ezt a folyamatot fel lehetett volna gyorsítani a videókártya CUDA támogatásával, azonban az én régebbi GPU-m ezt nem támogatta, így maradt a lassabb CPU-alapú számítás. A végül használatra kiválasztott modell nem tökéletes, de megfelelően működik. További finomhangolásra és tesztelésre már nem jutott idő, mert a betanítási folyamat rendkívül hosszadalmas, nekem pedig haladnom kellett a projekttel. Az alábbiakban bemutatom a betanító algoritmust.

```python
class ResNetEmbeddingWithClassifier(nn.Module):
    def __init__(self, num_classes: int):
        super().__init__()
        base = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)

        # Freeze most layers for transfer learning
        for name, param in base.named_parameters():
            if "layer1" in name or "layer2" in name:
                param.requires_grad = False

        num_feats = base.fc.in_features
        base.fc = nn.Identity()  # remove final classification head
        self.backbone = base

        # Embedding head
        self.embedding_head = nn.Sequential(
            nn.Linear(num_feats, 256),
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(256, 256)
        )

        # Classification head
        self.classifier = nn.Linear(256, num_classes)

    def forward(self, x):
        feats = self.backbone(x)
        embedding = F.normalize(self.embedding_head(feats), p=2, dim=1)
        logits = self.classifier(embedding)
        return embedding, logits
```

37. ábra PyTorch neurális háló modell

A teljes betanítás alapját a 37 mutatja. Ez az osztály definiálja a modell felépítését, vagyis azt, hogy milyen rétegekből álljon a háló, és hol történjen a tanulás. Először betöltöm az alapmodellt, a *ResNet50*-et, amely egy mély konvolúciós neurális háló, majd ebbe betöltöm az *ImageNet*-en előre betanított súlyokat. Így a tanítás nem teljesen a nulláról indul, hanem már egy olyan modellből, amely ismeri az alapvető vizuális mintázatokat - például éleket, textúrákat, formákat. A *ResNet* esetében a névben szereplő szám a háló mélységét jelöli, tehát azt, hogy hány rétegen keresztül tanul. A *ResNet50* jó kompromisszum: 50 rétegből áll, így mélyebb, mint például a *ResNet18*, de kevésbé nagy és nehézkes, mint a *ResNet152*. Általános irányelv, hogy kisebb mennyiségű tanítóadat esetén célszerű kisebb mélységű hálót választani, mert gyorsabban tanul, és kevésbé

hajlamos a túltanulásra. Nagyobb adatmennyiségnél viszont a mélyebb hálók jobban teljesítenek, mert több és részletesebb jellemzőt tudnak megtanulni. Bár az én adatkészletem viszonylag kicsi volt - 60 kupakosztályt hoztam létre, osztályonként legalább 5 képpel, így körülbelül 300 képen tanult a modell - a tesztjeim alapján mégis a *ResNet50* adott jobb eredményt, nem a kisebb mélységű modellek. Ezek után a modell legmélyebb két rétegét - amelyek az univerzális képi minták felismeréséért felelnek - lefagyasztom. Ez azt jelenti, hogy ezeket a rétegeket nem tanítjuk tovább, hanem megtartjuk az *ImageNet*-en előtanult súlyaikat. Ennek két előnye is van: egyrészt ezek a rétegek olyan általános jellemzőket tanulnak meg (például élek, textúrák, alakzatok felismerése), amelyek minden képre igazak, ezért nincs értelme újratanítani őket; másrészt a tanítás így gyorsabb és stabilabb lesz. Ezután eltávolítom a *ResNet50* eredeti fc kimeneti rétegét, mivel saját embedding kimenetet szeretnék létrehozni. A *self.embedding_head* rétegben hozom létre ezt az új kimenetet: az eredeti 2048 dimenziós *ResNet*-kimenetet 256 dimenziós vektorra csökkentem. Erre azért van szükség, mert a 2048 dimenzió túl nagy lenne a hasonlóság-számításhoz és a későbbi tároláshoz is. Ezután hozzáadok egy osztályozó fejet is, amely a cross-entropy veszteség kiszámításához szükséges. A *forward()* függvényben visszaadom a normalizált 256 dimenziós embeddinget - amelyet a *ResNet50* a képből kinyer, ezt azért normalizálom mert hasonlóságszámításnál a hossz nem számít csak az irány. Ezen kívül visszaadom a logitokat, amelyek az osztályozáshoz szükségesek.

```
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomRotation(180),
    transforms.ColorJitter(0.4, 0.4, 0.4, 0.2),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomVerticalFlip(p=0.5),
    transforms.RandomPerspective(distortion_scale=0.1, p=0.5),
    transforms.RandomApply([transforms.GaussianBlur(kernel_size=3, sigma=(0.1,
1.0))], p=0.2),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])
```

38. ábra Kód képek mesterséges módosítására

Nagyon fontos része az algoritmusnak amit a 38. ábra mutat. Mivel osztályonként csak kevés kupakfotót sikerült összegyűjteni, ráadásul ezek többsége web scrapinggel

származott más gyűjtőktől, akik általában jó minőségű, hasonló beállítású képeket készítenek, ezért az egyes kupakokhoz tartozó képek sokszor nagyon hasonlítottak egymásra. Annak érdekében, hogy a modell hibaállóbb és jobban általánosító legyen, szükség volt arra, hogy ugyanazon kupakról sokkal változatosabb bemenetet kapjon. Ennek megoldására alkalmaztam a 38. ábra által szemléltetett image augmentation eljárást. A rendszer betölti az eredeti képet, majd még az osztályozó modell elé kerülés előtt - a paraméterek alapján, véletlenszerűen - módosítja azt. Így az 5 eredeti képből lényegesen több, „különbözőnek tűnő" variáció állítható elő. A használt augmentációs műveletek közé tartozik többek között a véletlen forgatás, színmódosítás, tükrözés, perspektívaváltoztatás és elmosás (blur). Ez jelentősen javítja a modell robusztusságát és csökkenti a túltanulás esélyét.

```
for epoch in range(epochs):
    model.train()
    total_loss = 0.0

    for imgs, labels in dataloader:
        imgs, labels = imgs.to(device), labels.to(device)

        embeddings, logits = model(imgs)

        hard_triplets = miner(embeddings, labels)
        metric_loss = triplet_loss(embeddings, labels, hard_triplets)
        class_loss = ce_loss(logits, labels)

        loss = 0.3 * class_loss + 0.7 * metric_loss

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(dataloader)

    scheduler.step(avg_loss)
```

39. ábra A betanítás loop

Az adatok betöltése után elkezdődött maga a betanítás, amelyet a 39. ábra által szemléltetett kód hajtott végre. Az epoch, ahogy a 2.6.1-es fejezetben már említettem, azt jelenti, hogy hányszor menjen végig a modell a teljes adathalmazon. Én 20 epochot

állítottam be, egyrészt mert a betanítás hosszadalmas volt, másrészt a kis adathalmaz miatt el akartam kerülni a túltanulást. A futások alapján azt tapasztaltam, hogy a veszteség (loss) értéke nem csökken tovább, vagy nagyon lassan változik 20 epoch után, ilyenkor a modell már nem tanul új információt, ezért célszerű a futást leállítani. A kód batch-enként dolgozik, vagyis a megadott batch-méretbe beleférő képeket és címkéket adja a modellnek. Az előzőekben ismertetett embedding és osztályozó *ResNetEmbeddingWithClassifier* osztály feldolgozza a batch-et, és visszaadja a képek embeddingjét és a *logits*-okat. Ezután kiszámítom a triplet-loss-t, amelyet a 2.7.2-es fejezetben ismertettem, valamint a cross-entropy loss-t. A két loss-t meghatározott súllyal összeadom, így kapom meg a végső veszteség értékét. A tesztek alapján ez a súlyozás biztosította a modell legjobb pontosságát. Végül frissítem a súlyokat, és finomhangolom a learning rate-et. A learning rate és a gradiens együtt határozza meg a súlyváltozás mértékét.

4.3.3 Tárgyfelvétel csővezeték

Az utolsó fejezetben bemutatom a legfontosabb, bonyolultabb csővezetéket (pipeline), vagyis azt a folyamatot, amely során a felhasználó kép alapján tud új tárgyat felvenni a kollekciójába. Ez a folyamat különösen fontos, mert ekkor alkalmazom az előző fejezetekben ismertetett, betanított modelleket.

```
public record ImageCheckResponse(Guid Id, string State, int Quantity,
List<ItemProvided> Similars);

public record ImageCheckResult(Guid Id, string State, int Quantity, List<Guid>
Similars);

public record ImageCheckPythonServiceResponse(string Status,
List<ImageCheckResult> Response);

[HttpPost("ImageCheck/{id}")]
public async Task<ActionResult<IEnumerable<ImageCheckResponse>>>
UploadImageCheckTest(Guid id, IFormFile file)
```

40. ábra Kép ellenőrző API függvény fejléc

A frontend oldalon, amikor a felhasználó lefotózza a tárgyat, vagy kiválaszt egy képet a galériából tárgyfelvételkor, először az a hívás fut le, amit a 40. ábra mutat. A paraméterként átadott *id* jelzi, hogy a felhasználó melyik kollekciójához tartozó tárgyakkal kell összehasonlítani a képen esetlegesen megtalált tárgyakat, vagyis melyik kollekcióba szeretnénk majd felvenni az új elemet.

```python
@app.post("/image_check/{collection_type_id}/{collection_id}")
async def image_check(
    collection_type_id: str,
    collection_id: str,
    file: UploadFile = File(...)):
```

41. ábra Python Service kép ellenőrző API függvény fejléc

Ezután a backend meghívja a Python Service kép ellenőrző API-t. Az API-t a 41. ábra szemlélteti. A hívás egy további paramétert is vár: a *collection_type_id*-t, mert ebből tudja a service, hogy melyik kollekció típus betanított modelljeit kell használni - ez az „univerzális” megoldáshoz szükséges. A service elvégzi a kép elemzését: a betanított modell kivágja a képből az észlelt tárgyakat, majd a másik modell meghatározza a kivágott képek embeddingjét. Minden kivágott tárgy ideiglenes, véletlenszerű azonosítót kap, amivel nyomon követhető, amíg nem kerül be a kollekcióba. A kivágott képek és a hozzájuk tartozó embedding fájlok 10 percig maradnak tárolva, így ha a felhasználó újként szeretné felvenni a tárgyat, az ideiglenes adatok felhasználhatók, és nem kell újraszámolni őket. Ha a 10 perc alatt nem történik változás, a fájlok automatikusan törlődnek.

```python
[
    {
        "id": "a3f1c8d2-7e45-4b6e-9f3c-2d1e5a8b9f10",
        "state": "found",
        "quantity": 7,
        "similars": [
            "b2e4f9a1-3c5d-4d8f-9a7b-1e2f3c4d5b6a",
            "c1d2e3f4-5a6b-7c8d-9e0f-1a2b3c4d5e6f"
        ]
    },
    {
        "id": "d4e5f6a7-8b9c-0d1e-2f3a-4b5c6d7e8f90",
        "state": "not found",
        "quantity": 3,
        "similars": []
    }
]

{"status": "ok", "response": response}
```

42. ábra Python Service kép ellenőrző API függvény példa válasz

A 42. ábra a fent ismertetett Python Service API hívásának példa válaszát mutatja JSON objektum formájában. Itt minden detektált tárgyat visszaadunk az ideiglenes azonosítójával, valamint a *state* mezővel, amely két értéket vehet fel:

- *found*: a rendszer talált ilyen tárgyat a vektoradatbázisban és így a kollekcióban is,
- *not found*: nincs találat, vagyis új tárgyról van szó.

A *quantity* mező akkor érdekes, ha egy képen több ugyanolyan tárgy található; ilyenkor a rendszer a tárgyat egyszer küldi vissza, de a *quantity* jelzi, hogy mennyi van belőle a képen.

A *similars* tömb tartalmazza a hasonló tárgyak azonosítóit. Ez a mező mindkét *state* esetén értelmezett:

- *found* esetén az első azonosító a megtalált tárgyé, a többi pedig a hasonló tárgyakat jelöli,
- *not found* esetén a hasonló tárgyakat jelöli

Így a felhasználó mindkét esetben dönthet arról, hogy elfogadja a mesterséges intelligencia predikcióját, vagy másik hasonló tárgyat választ, vagy újként veszi fel a kollekcióba.

```
public record ItemIds(Guid TempId, Guid Id);
public record InsertedItem(ItemIds ItemIds, ItemProvided ItemProvided);
public record UpsertItemRequest(Guid Id, ItemRequired Item);

[HttpPost("Upsert")]
public async Task<ActionResult> Upsert([FromBody] UpsertItemRequest[] request)
```

43. ábra Tárgyfeltöltés API függvény fejléc

Miután a backend visszaküldte a felhasználónak a Python Service által visszaadott adatokat, a felhasználó eldöntheti, hogy miként veszi fel a tárgyakat. Az új tárgyakhoz adatokat rendelhet, majd a *Finish* gombra kattintva megtörténik a feltöltés. Az adatbázisban történő módosítás a 43. ábra által szemléltetett API függvényen keresztül történik, és egy nagy tranzakcióként zajlik: először létrejönnek vagy frissülnek az adatbázisban a rekordok, majd az ideiglenes kép fájlokat mentem át a végleges tárolóba az ideiglenes azonosító alapján, és végül meghívom a vektoradatbázisba mentést. Ezt a sorrendet azért tartom, mert a tárgy azonosítóját csak az adatbázisban létrejött rekord után lehet használni. A tranzakció biztosítja, hogy hiba esetén minden módosítás

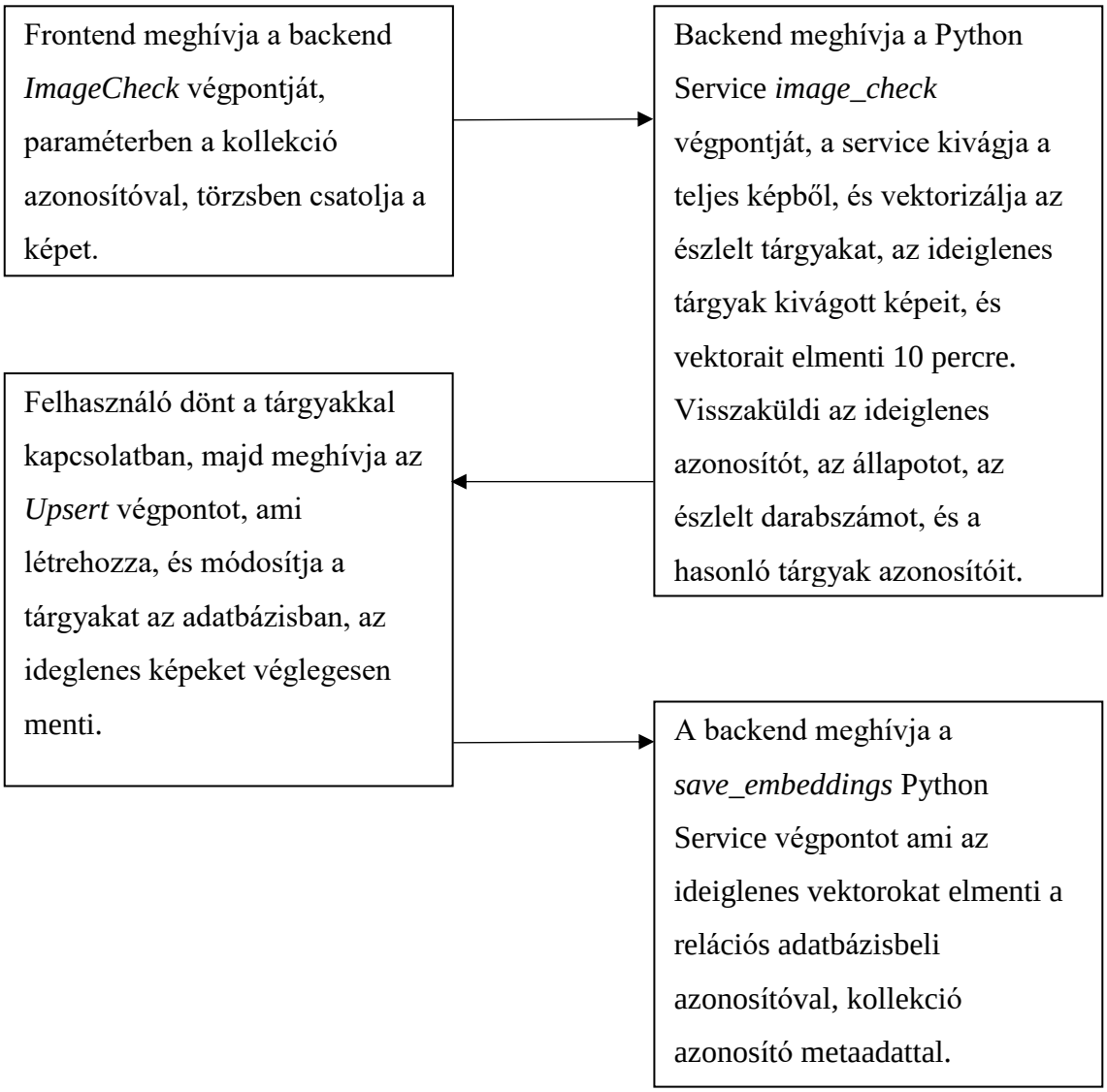
visszavonható legyen (rollback). A Python Service-nek az *ItemIds* tömböt küldöm vissza, amely tartalmazza mind az ideiglenes, mind az adatbázisbeli azonosítókat.

```
@app.post("/save_embeddings/{collection_id}")
async def save_embeddings(
    collection_id: str,
    items: List[ItemIds] = Body(...)
):
```

44. ábra Vektoradatbázis feltöltés API függvény fejléc

Az új tárgyak embeddingjét el kell menteni a vektoradatbázisba, hogy a következő tárgyfelvétel során a Python Service már ezekkel szemben is ellenőrizhesse a hasonlóságot az adott kollekcióban. Az API hívás függvényt a 44. ábra szemlélteti. A vektoradatbázisba a 3.3-as fejezetben ismertetett séma szerint kell elmenteni az adatokat: a tárgy vektora kerül a vektorhoz, az *id* a relációs adatbázisbeli azonosítót tartalmazza, a metaadat pedig a kollekció azonosítója, ahová a tárgy tartozik. A rendszer ekkor az ideiglenesen elmentett embeddinget az ideiglenes azonosító alapján átmásolja a vektoradatbázisba a megadott adatokkal.

Az alábbiakban összefoglalom diagrammon keresztül a csővezeték működését:



5 Projektértékelés

6 Összefoglaló

Irodalomjegyzék

[1] Microsoft, „Overview of ASP.NET Core," 2025. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/overview?view=aspnetcore-9.0. [Hozzáférés dátuma: 6 november 2025].

[2] Microsoft, „Attributes," 2025. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/csharp/advanced-topics/reflection-and-attributes. [Hozzáférés dátuma: 7 november 2025].

[3] Microsoft, „Relationship navigations," 2025. [Online]. Available: https://learn.microsoft.com/en-us/ef/core/modeling/relationships/navigations. [Hozzáférés dátuma: 6 november 2025].

[4] Expo, „Expo Docs," 2025. [Online]. Available: https://docs.expo.dev. [Hozzáférés dátuma: 7 november 2025].

[5] React Native Elements, „React Native Elements Docs," 2025. [Online]. Available: https://reactnativeelements.com/docs. [Hozzáférés dátuma: 7 november 2025].

[6] Label Studio, „Label Studio Docs Template Gallery," 2025. [Online]. Available: https://labelstud.io/templates. [Hozzáférés dátuma: 7 november 2025].

[7] Label Studio, „Label Studio Docs Export annotations and data from Label Studio," 2025. [Online]. Available: https://labelstud.io/guide/export.html. [Hozzáférés dátuma: 7 november 2025].

[8] B. Joel, „What is embedding?," [Online]. Available: https://www.ibm.com/think/topics/embedding. [Hozzáférés dátuma: 8 november 2025].

[9] M. Leonie és H. Zain, „A Gentle Introduction to Vector Databases," 1 augusztus 2023. [Online]. Available: https://weaviate.io/blog/what-is-a-vector-database. [Hozzáférés dátuma: 8 november 2025].

[10] S. Sindhu, „Machine Learning Fundamentals: Cosine Similarity and Cosine Distance," 26 május 2021. [Online]. Available: https://medium.com/geekculture/cosine-similarity-and-cosine-distance-48eed889a5c4. [Hozzáférés dátuma: 8 november 2025].

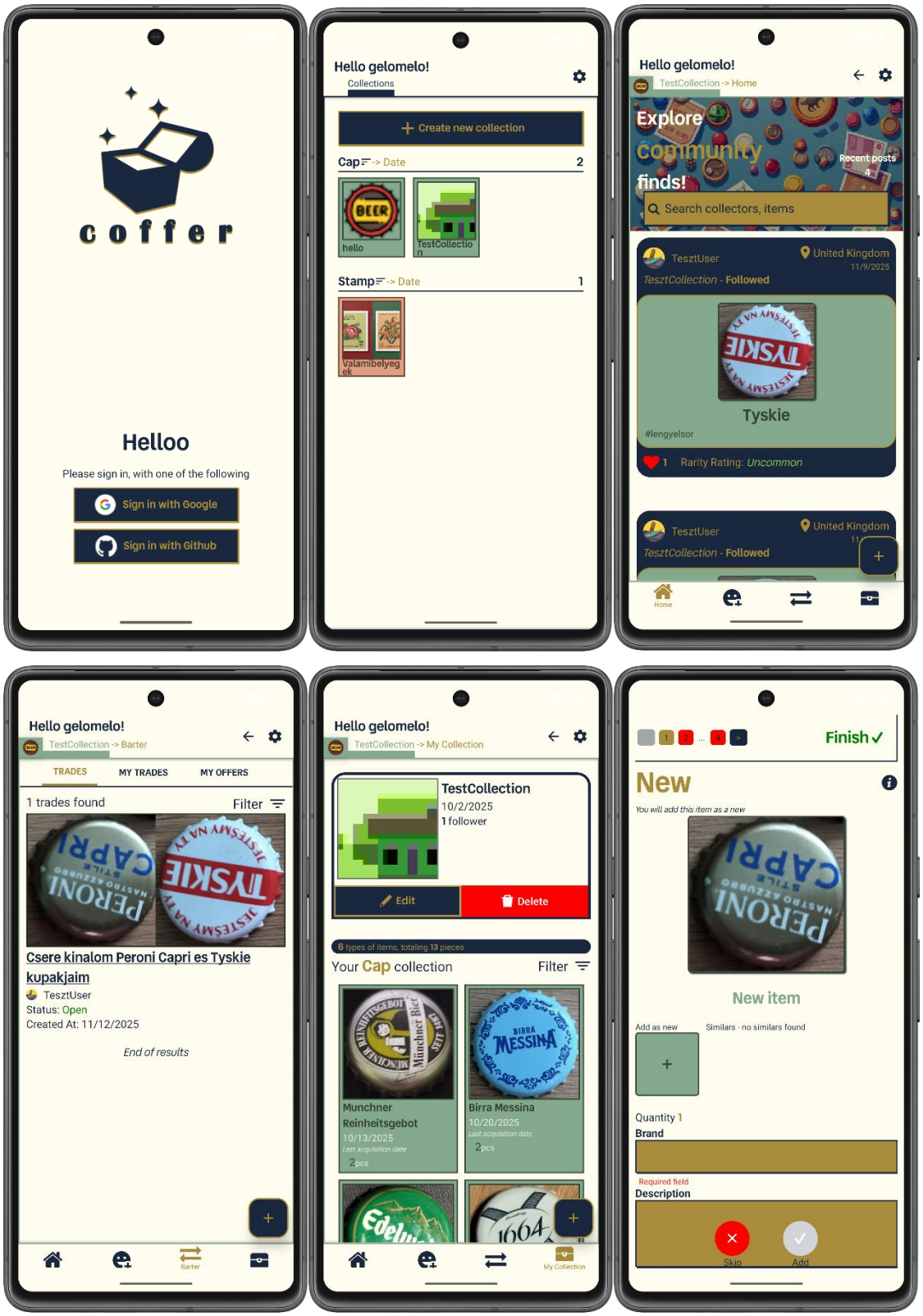
[11] K. Hareesh, „Digging Deeper into Metric Learning — Loss Functions," 9 november 2019. [Online]. Available: https://medium.com/slyce-engineering/digging-deeper-into-metric-learning-loss-functions-29d89edfe200. [Hozzáférés dátuma: 8 november 2025].

[12] Microsoft, „Tutorial: Create a controller-based web API with ASP.NET Core," 2025. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-web-api/_static/architecture.png?view=aspnetcore-9.0. [Hozzáférés dátuma: 7 november 2025].

[13] P. S. Jon, „The Reformed Programmer," [Online]. Available: https://www.thereformedprogrammer.net/wp-content/uploads/2020/07/ConfigurationHeader.png. [Hozzáférés dátuma: 6 november 2025].

Függelék A – Képernyőképek az applikációból

Login, *Collections*, *Feed*, *Barter*, *My Collections*, *New Items* oldalak



Függelék B – Teljes ER diagram